Cairo University
Faculty of Computers and Artificial Intelligence
Artificial Intelligence Department
Information Systems Department

Seed Bank

Quality Intelligence Platform

Graduation Project Final Documentation

Prepared by:

Omar Ezz-ElDein Abdullah	20220228
Youssef Ahmed Awad	20220385
Ali Abdulrahman Mohamed	20220213
Youssef Tarek Ali	20220397
Mohamed Amr Mahmoud	20220302

Supervised by:

Dr. Eman Ahmed
Dr. Ali Zidane
Dr. Heba Sherif
Dr. Ghada Dahy

Academic Year: 2025/2026

Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
CAS	Compare-And-Set
CBAM	Convolutional Block Attention Module
CDC	Change Data Capture
CLI	Command Line Interface
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset format)
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DWH	Data Warehouse
ERD	Entity-Relationship Diagram
FCAI	Faculty of Computers and Artificial Intelligence
FPN	Feature Pyramid Network
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
IoU	Intersection over Union
JSON	JavaScript Object Notation
JWT	JSON Web Token
LRU	Least Recently Used
MIME	Multipurpose Internet Mail Extensions
ML	Machine Learning
OAuth	Open Authorization
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
ORM	Object-Relational Mapping
OTel	OpenTelemetry
OTLP	OpenTelemetry Protocol
RBAC	Role-Based Access Control
R-CNN	Region-based Convolutional Neural Network
REST	Representational State Transfer
RTL	Right-To-Left
SPA	Single-Page Application
UI	User Interface
YAML	YAML Ain't Markup Language
YOLO	You Only Look Once

Abstract

English Abstract

Seed quality is a major factor in germination and final yield, yet it is still judged by hand: a grader pours out a sample, spreads it on a tray, and sorts the seeds into good and bad piles. That process is slow and subjective: it covers only a small fraction of a large lot, and two graders rarely agree on the borderline seeds. Seed Bank is a platform that automates this judgement. A user photographs a batch of seeds; an object detector locates every seed, a per-crop classifier grades each one as good or bad, and the system reports the seed count, the good-rate, and a breakdown by crop. The two crops supported today are coffee and maize. The backend is an asynchronous service with a layered architecture, a model registry, offline model evaluation, and a foreign-key chain that traces every seed label back to the exact model version that produced it. One backend serves three clients: a web app, a bilingual mobile app with a realtime camera, and a documented HTTP API. Training the detector exposed a data problem, because public seed datasets are almost all single-seed images while a detector needs annotated multi-seed images that are expensive to label by hand. The companion tool MultiSeedGen closes this gap by compositing single-seed crops into annotated multi-seed scenes, with reproducible and leakage-safe output. The result is a working, traceable platform together with the data generator that makes its detector trainable.

Arabic Abstract

تُعدّ جودة البذور عاملاً مهمّاً في نسبة الإنبات وفي المحصول النهائي، ومع ذلك ما زال تقييمها يجري يدويّاً: يسكب المُصنّف عيّنة من البذور وينشرها على صينية ويفرزها إلى سليمة وتالفة. هذه العملية بطيئة، ولا تشمل سوى جزء ضئيل من الدُّفعة الكبيرة، وتختلف نتائجها من شخص إلى آخر. نظام «Seed Bank» منصّة تُؤتمت هذا الحكم؛ إذ يلتقط المستخدم صورةً لدُفعة من البذور، فيحدّد نموذجٌ للكشف عن الأجسام موضع كلّ بذرة، ثمّ يصنّف نموذجٌ خاصّ بكلّ محصول كلَّ بذرة إلى سليمة أو تالفة، ويعرض النظام عدد البذور ونسبة السليم وتوزيعاً بحسب نوع المحصول. والمحصولان المدعومان حاليّاً هما البُنّ والذُّرة. بُني النظام الخلفيّ بمعماريّة طبقيّة غير متزامنة، مع سجلّ للنماذج، وتقييم للنماذج دون اتصال، وسلسلة مفاتيح خارجيّة تتيح تتبّع كلّ نتيجة إلى إصدار النموذج الذي أنتجها. وتخدم هذه الواجهة الخلفيّة ثلاثة تطبيقات: تطبيق

ويب، وتطبيق جوّال ثنائيّ اللغة بكاميرا فوريّة، وواجهة برمجيّة موثّقة. وقد كشف تدريبُ نموذج الكشف عن مشكلة في البيانات، لأنّ معظم مجموعات البيانات المتاحة تضمّ بذرةً واحدة في الصورة، بينما يحتاج الكاشف إلى صور متعدّدة البذور مُوسَمة يصعب إعدادها يدويّاً. تسدّ الأداة المرافقة «MultiSeedGen» هذه الفجوة بتركيب قُصاصات البذور المفردة في مشاهد متعدّدة البذور مُوسَمة تلقائيّاً، بنتائج قابلة لإعادة الإنتاج وخالية من تسرّب بيانات التحقّق.

Chapter 1

Introduction

1.1 Background and Motivation

Seed quality decides a large part of what a farmer harvests. A lot of seed that looks fine to the eye can still carry a high fraction of broken, immature, discoloured, or diseased grains, and that fraction sets the germination rate and the final yield. Before a lot is planted, traded, or priced, someone has to judge how good it is. In most of the supply chain that judgement is still made by a human grader who pours out a sample, spreads it on a tray, and sorts the seeds by hand into good and bad piles, counting as they go.

Manual inspection has two problems that are hard to fix by training people better. It is slow: a careful grader works through a few hundred seeds before fatigue sets in, which is a tiny sample of a multi-tonne lot. It is also subjective. Two graders looking at the same tray will disagree on the borderline seeds, and the same grader will drift over a long shift. The result is a quality number that is neither repeatable nor auditable, which is a weak basis for a price or a planting decision.

Computer vision fits this task well. The decision a grader makes, ”is this individual seed good or bad”, is a per-object visual classification, and the wider job of finding every seed in a tray is object detection. Both have matured to the point where they run on commodity hardware and beat a tired human on consistency. Surveys of deep learning in agriculture report strong results across crop and produce inspection tasks once enough labelled data is available [1], and the same convolutional models that classify plant disease from leaf photographs transfer cleanly to grading the seeds themselves [2]. An automated grader does not get tired, scores every seed in the photographed sample, and produces the same answer twice for the same image.

This project, *Seed Bank*, builds that automated grader as a working platform rather than a notebook demo. It supports two crops today, coffee and maize, chosen because they are economically important and visually distinct enough to exercise the two-crop routing in the pipeline. A user photographs a batch of seeds; the system locates each seed, scores its quality, and reports the aggregate good-rate, seed count, confidence distribution, and a per-crop breakdown. One

backend serves both audiences: farmers in the field, through a web and a mobile app, and the AI developers who run the models behind the product.

1.2 Problem Definition

Two linked problems sit at the centre of this project. The first is the shape of the machine-learning task itself. The second is a data problem that the first one exposes, and it is the harder of the two.

1.2.1 Grading a tray photo is two ML problems in sequence

A photo of a tray holds many seeds at once, so grading it cannot be a single classification. It splits into two stages that run one after the other.

The first stage is **detection**: given the whole image, find where every seed is. An object detector returns a bounding box, a confidence score, and a class label for each seed it locates, turning one photo into N located seeds. In Seed Bank the detector's class identifies the crop, coffee or maize, because a single tray can mix both and the next stage has to know which kind of seed it is looking at.

The second stage is **classification**: for each detected seed, decide whether it is good or bad. The system crops the bounding box out of the original image and feeds that crop to a binary classifier that returns `good`/`bad` plus a confidence. There is one classifier per crop type, so a coffee crop goes to the coffee classifier and a maize crop to the maize classifier. The data fans out along the way: one image becomes N detections, which become N quality labels, and the aggregate report is computed from that graph.

The two stages are genuinely different models with different training needs, and keeping them separate is what lets a detector and a classifier be versioned and swapped on their own. It also means the two stages need different training data, which is where the second problem appears.

1.2.2 The dataset gap

The two stages do not have the same data appetite, and that asymmetry is the problem this project had to solve before any of the platform work mattered.

The classifiers are easy to feed. A binary good/bad classifier trains on **single-seed** crops, one seed centred in each image with a single label. Public seed datasets are full of exactly this kind of image, because a single seed on a plain background is cheap to photograph and trivial to label. Coffee and maize both have usable single-seed corpora.

The detector is the problem. An object detector needs **multi-seed** images, many seeds scattered in one frame, each one annotated with a bounding box. That data barely exists in public. Almost every published seed dataset is single-seed, which is useless for training a detector

because there is nothing to locate, no second object, no clutter, no overlap. Building a real multi-seed detection set by hand means photographing trays of scattered seeds and drawing a box around every grain in every image, hundreds of boxes per photo, thousands of photos. That labelling effort is the bottleneck, and it is expensive and slow enough that it stops most teams before they start.

So the situation is lopsided. The data needed to train the quality classifiers is abundant, and the data needed to train the detector is scarce and costly to produce. Closing that gap is what motivated the second half of this project, *MultiSeedGen*: a tool that takes the single-seed crops we already have and synthesises annotated multi-seed images from them, so the detector can be trained without hand-labelling thousands of trays.

1.3 Project Objectives and Proposed Solution

The objectives follow directly from the two problems above:

- Build an end-to-end platform that takes a photo of a seed batch and returns a grading report, running the detect-then-classify pipeline for coffee and maize.
- Make every result traceable: any seed label must be linkable back to the exact model version and weights that produced it.
- Serve the same backend to three clients, a web app, a mobile app, and a programmatic API, so the product reaches both farmers and developers.
- Give the ML side real tooling: a model registry with a promotion lifecycle, a production-model resolver that selects the promoted production model for each segment, and offline evaluation of any model against labelled datasets.
- Close the detection-data gap by generating annotated multi-seed training images from single-seed crops.

The solution has two parts that match the two groups of objectives.

Seed Bank is the platform. Its backend is a FastAPI service that orchestrates the two-stage pipeline: a request uploads images, a detector finds the seeds, each seed is cropped and classified, and the results are persisted with a hard foreign-key chain from `seed_detection` to `inference` to `model_artifact`, which is what makes a result traceable. Around that pipeline sits a model registry that moves weights through a `registered` $\rightarrow$ `staging` $\rightarrow$ `production` $\rightarrow$ `archived` lifecycle, a model resolver that selects the promoted production model for each segment with a global fallback, and an experiment runner that scores models offline against ground-truth datasets. Three clients consume one API: a React web app, an Expo / React Native mobile app with a realtime camera, and the documented HTTP API itself.

MultiSeedGen is the data generator. It cuts individual seeds out of source photos, then composites many of them onto realistic backgrounds with domain-matched lighting and camera degradation, and exports detection and segmentation datasets in YOLO, YOLO-seg, and COCO formats with full provenance. The cut-and-paste idea, pasting real object crops onto varied backgrounds to synthesise detection training data, is a known and effective way to produce labelled examples cheaply [3], and the deliberate randomisation of lighting, scale, rotation, and noise follows the domain-randomisation principle of forcing a model to generalise by training it on a wide spread of synthetic variation [4]. Because every seed is placed by the tool, its bounding box and mask are known exactly, so the annotations come for free and the output is byte-reproducible for a fixed configuration and random seed.

1.4 Project Scope and Limitations

In scope. The platform grades two crops, coffee and maize, and is built so a third crop is added by registering a new detector class and a matching classifier rather than by rewriting the pipeline. It serves three clients (web, mobile, API), resolves the promoted production model per segment with a global fallback, runs offline evaluation of any registered model against labelled datasets, and ships an observability stack covering structured logs, Prometheus metrics, OpenTelemetry traces, and Sentry error reporting. MultiSeedGen covers the full single-seed-to-multi-seed generation path with several segmentation backends and reproducible output.

Out of scope and current limits. The system is a working platform with a known unfinished tail, recorded honestly so the reader is not misled:

- **Upload.** Image upload to `POST /analyze` is multipart-only. Presigned direct-to-storage upload and batch cancellation were specified but never built.
- **Analytics ingestion.** The ClickHouse warehouse is populated by an application-level dual-write rather than true change-data-capture, even though Postgres is configured with `wal_level=logical` so CDC can be added later.
- **Observability is opt-in.** OpenTelemetry tracing and Sentry are no-ops unless their endpoints are configured, so the default dev stack does not run them.
- **Some endpoints lack routes.** A few features have a service and a schema but no wired route yet (password change, experiment-results fetch, presigned upload).
- **The synthetic-to-real domain gap.** A detector trained only on MultiSeedGen output learns the statistics of synthetic scenes, which never match real trays perfectly. Narrowing that gap with augmentation and realistic degradation is the tool's central design concern, but it cannot be assumed closed.

1.5 Development Methodology

The platform was not written in one pass. It was delivered iteratively, in phases, each one a usable slice of the system: scaffolding the FastAPI app, laying the 18-table async schema, wiring the infrastructure clients and health probes, building auth, then the ML platform, then the inference path, experiments, the data warehouse, and observability. That phasing is recorded in the project's revamp-status document, and it meant each layer was exercised before the next was stacked on it.

The backend follows clean-architecture principles [5], enforced as five rules rather than left as good intentions:

- **Async end-to-end.** FastAPI, async SQLAlchemy, and async clients throughout, so nothing blocks the event loop.
- **Layered.** Every request flows `routers` → `services` → `repositories` → ORM. Routers stay thin, business logic lives in services, and only repositories touch the ORM.
- **Pydantic at the boundaries.** Every request and response is a validated schema; the ORM never leaks past the service layer.
- **One central `Settings`.** All configuration comes from a single settings object fed by environment variables or file secrets; code never reads the environment directly.
- **Model traceability.** The `seed_detection` → `inference` → `model_artifact` chain is enforced with foreign keys, not convention.

Work was managed with a git-based workflow. A pre-commit hook runs the formatter and linter (ruff), the strict type checker (mypy), and a secret scanner (gitleaks) before anything is committed, and the same checks are intended to run as CI gates. The test suite is a pytest pyramid with unit, integration, and end-to-end tiers, the integration tier using real Postgres and Redis through testcontainers. The honest state of that suite, including the parts that are not yet hermetic, is reported in the testing chapter.

1.6 Tools and Technologies Used (Software and Hardware)

The stack is deliberately conventional where it can be, so that each choice is a well-understood tool with a known failure mode rather than a novelty. The backend is built on FastAPI with async SQLAlchemy [6], [7]; model experiments persist their metrics to PostgreSQL alongside a Markdown report stored in MinIO, with no external tracking server; the web client is React [8] and the mobile client is Expo / React Native [9]; and the whole system runs from a single Docker Compose file [10]. Table 1.1 groups the full set by area.

Table :1.1 Tools and technologies used, grouped by area.

Area	Tools
Backend	Python 3.12, FastAPI, async SQLAlchemy 2 with asyncpg, Pydantic v2 and pydantic-settings, Alembic migrations.
Async & ML	Celery with a Redis broker, PyTorch and torchvision, Ultralytics YOLO, Roboflow inference SDK, Pillow and OpenCV (headless).
Datastores	PostgreSQL 16 (OLTP), Redis 7 (cache, broker, rate-limit store), MinIO (object store), ClickHouse (analytics warehouse).
Observability	structlog, Prometheus, Grafana, OpenTelemetry, Sentry.
Web	React 18, TypeScript, Vite, Tailwind CSS, TanStack Query.
Mobile	Expo SDK 52, React Native 0.76, expo-camera, React Navigation.
Infrastructure	Docker and Docker Compose, a multi-stage Dockerfile, the `uv` dependency manager, and Make as the developer entrypoint.

The split into many small runtime images is intentional. The API and the lightweight worker carry no PyTorch at all, because torch and torchvision pull in roughly 1.6 GB and only the inference worker ever runs a model. That keeps the API container small and the import cost of a routine background job low.

On the hardware side, development was done on a workstation with an NVIDIA RTX 3060 GPU. The GPU was used for two things: training the detector and classifier models, and accelerating the rembg (U^2-Net) segmentation step in MultiSeedGen when cutting seeds out of source photos. The Seed Bank platform itself runs CPU-only in the development stack; the production Compose overlay swaps the inference worker to a CUDA 12.4 GPU runtime with a device reservation, so the same code path runs on a GPU in production without change.

1.7 Project Timeline / Gantt Chart

The work was scheduled across the 2025/2026 academic year and ran in the phased order described in the methodology section above. The early part of the year went to problem framing and to MultiSeedGen, because the platform's detector could not be trained until synthetic detection data existed. The middle of the year built the backend phase by phase, from the application scaffold and database schema through auth, the ML platform, and the inference path. The later phases added experiments, the analytics warehouse, and observability, in parallel with the web and mobile clients and the bilingual localization work. The final stretch was reserved for hardening, the production Compose overlay, and this report. Figure 1.1 shows the schedule.

1.8 Report Organization

The rest of this report is organised as follows.

Chapter 2, Related Work, reviews the background this project builds on: object detectors

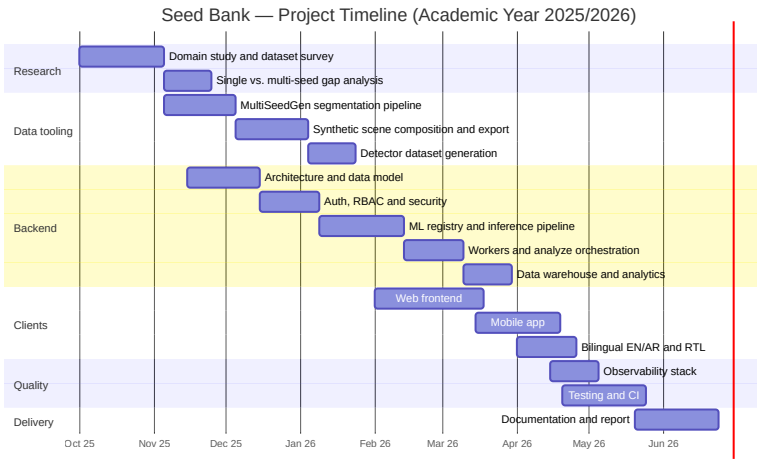

Figure :1.1 Project timeline (Gantt chart).

such as Faster R-CNN and YOLO and the feature-pyramid and attention ideas behind them, image classifiers from the ResNet family, the synthetic-data and augmentation literature that justifies MultiSeedGen, and prior applications of computer vision in agriculture.

Chapter 3, System Analysis, sets out the requirements. It defines the user roles, the functional and non-functional requirements, and the use cases the platform has to support, and frames them against the two problems from this chapter.

Chapter 4, System Design, describes the architecture: the layered backend, the data model and its traceability chain, the two-stage inference pipeline with its model registry and model resolver, the data warehouse, and the three clients.

Chapter 5, Implementation, covers how the design was actually built, the model builders and inference backends, the Celery orchestration of the detect-then-classify flow, MultiSeedGen's generation pipeline, and the deployment setup.

Chapter 6, Testing and Evaluation, reports the test pyramid and quality gates, the model evaluation results, and an honest account of the suite's current gaps.

Chapter 7, Conclusion and Future Work, summarises what was delivered against the objectives and lays out the unfinished tail and the next steps, including true change-data-capture, the missing endpoints, and narrowing the synthetic-to-real domain gap.

Chapter 2

Related Work

This chapter reviews the research and engineering that Seed Bank builds on. It moves from the application domain (computer vision for seed and grain quality) through the model families the platform uses (object detectors and attention classifiers), into the area that mattered most for getting the project off the ground (synthetic data for training detectors without hand-labelling), and finally to the operational side (managing many model versions in production). The last section sets prior work against what this project actually delivers and explains where the two differ.

2.1 Automated Seed and Grain Quality Assessment

Grading seeds and grain by eye is slow and inconsistent between graders, so the problem has attracted machine vision for a long time. Early systems took a photograph of seeds against a controlled background, segmented each kernel with classical image processing (thresholding, edge detection, morphological operations), and then fed hand-engineered features to a shallow classifier. Typical features were colour histograms, geometric descriptors such as area, perimeter, eccentricity and aspect ratio, and texture statistics from co-occurrence matrices. These pipelines work when the imaging rig is fixed and the seeds are well separated, because every stage assumes a clean, predictable input. They degrade quickly once lighting drifts, the background changes, or kernels touch and overlap, since the segmentation step has no model of what a seed looks like and treats any connected blob as one object.

Convolutional networks removed the need to design features by hand. Instead of choosing which colour and texture statistics matter, a CNN learns them from labelled examples, which is what makes deep models so much more tolerant of the messy conditions that break classical pipelines. The survey by Kamilaris and Prenafeta-Boldú [1] catalogues this shift across agriculture: yield prediction, weed and crop classification, fruit counting, and quality grading all moved from hand-crafted features to learned representations over a few years, with consistent accuracy gains where enough labelled data was available. The recurring caveat in that survey

is data. Deep models need many labelled images, and in agriculture the labels are expensive because they require an agronomist, not a crowd worker.

Plant pathology shows the same pattern at finer detail. Mohanty et al. [2] trained CNNs to recognise crop diseases from leaf photographs and reached high accuracy on a curated dataset, but the same paper reports that accuracy fell sharply on images taken under conditions different from the training set. That gap between a clean benchmark and field photographs is the central practical problem for any agricultural vision system, and it is the reason this project treats data variety as a first-class concern rather than an afterthought.

Two design lessons carry over into Seed Bank. First, quality assessment of a photograph that contains many seeds is really two problems, not one: finding each seed, then judging it. Classical single-stage classifiers conflate the two and fail on cluttered images. Second, the quality bottleneck is labelled data, especially for the detection stage, where every seed in every image needs a bounding box. Both lessons shape the architecture described in the following sections.

2.2 Object Detection Architectures

The first stage of the Seed Bank pipeline locates every seed in an image and assigns it a crop class. This is object detection, and the field has settled into two broad families that trade accuracy against speed.

2.2.1 Two-stage detectors

Two-stage detectors first propose regions that probably contain an object, then classify and refine each proposal. Faster R-CNN [11] is the reference design: a Region Proposal Network shares convolutional features with the detection head and generates candidate boxes cheaply, replacing the slow external proposal methods used by earlier R-CNN variants. The shared backbone makes the whole detector trainable end to end and fast enough to be practical.

A backbone produces strong features at one scale, but seeds in a photograph vary in size depending on how close the camera is and how the kernels are arranged. The Feature Pyramid Network [12] addresses this by combining the coarse, semantically strong features from deep layers with the fine, high-resolution features from shallow layers. This gives the detector a consistent representation across scales at little extra cost. The detection model wired into Seed Bank is torchvision's `fasterrcnn_resnet50_fpn`: a ResNet-50 backbone [13] with an FPN neck and a three-class head (`background`, `coffee`, `maize`). This combination is a sensible default when detection accuracy matters more than raw throughput, which fits a quality-grading workload where each photo is analysed once and the result is stored.

2.2.2 One-stage detectors

One-stage detectors skip the proposal step and predict boxes and class scores directly from a single pass over the image. YOLO [14] introduced the idea by framing detection as a single regression over a grid, trading some localisation accuracy for a large speed gain. Later versions of the family closed much of the accuracy gap; the Ultralytics YOLOv8 release [15] is a widely used current implementation with strong tooling for training and export. Seed Bank does not commit to either family. Its inference layer exposes a YOLO backend (`ultralytics_yolo`) alongside the torchvision Faster R-CNN backend, so a deployment can run a fast one-stage detector where latency is the priority, for example a conveyor-belt scanning station, or the more accurate two-stage detector elsewhere, without changing any calling code.

2.2.3 Benchmark datasets

Both families are measured against shared public benchmarks, which is what makes results comparable across papers. ImageNet [16] provides the large classification dataset that backbones are pretrained on, and COCO [17] provides the detection benchmark with its standard mean average precision protocol. These datasets matter to Seed Bank in two concrete ways: the ResNet backbones it uses are ImageNet-pretrained, and the synthetic data generator described in Section 2.4 writes its annotations in COCO format, so generated datasets drop straight into standard detection training code.

2.3 Attention in Image Classfiiers

The second stage of the pipeline takes each detected seed, crops it from the original image, and decides whether that single seed is good or bad. This is binary image classification on a small crop, and the architecture is a ResNet-18 [13] backbone with a Convolutional Block Attention Module [18].

Residual networks made very deep classifiers trainable by adding identity skip connections, so gradients reach early layers without vanishing [13]. ResNet-18 is the shallow member of that family, small enough to run many crops per image quickly while still benefiting from ImageNet pretraining. For a per-seed classifier that may be invoked dozens of times per photograph, the small backbone is the right trade.

Attention refines what a fixed backbone attends to. CBAM [18] is a lightweight module that learns two attention maps in sequence: a channel map that reweights which feature channels matter, and a spatial map that reweights which positions matter. It plugs into an existing backbone with almost no parameter overhead and consistently improves classification accuracy in the original study. In the Seed Bank classifier the CBAM block is inserted after the final residual stage (`layer4`). The network then applies hybrid pooling that concatenates global-average and global-max pooled features (512 plus 512 into 1024 features) before a single-logit head, and a

sigmoid converts the logit into a good/bad score. The attention block helps the classifier focus on the discriminative part of a seed crop, such as a cracked or discoloured region, rather than spreading its response over the whole box. A separate classifier is trained for each crop type, which is why the detector groups its boxes by seed type before the classification stage runs.

2.4 Synthetic Data and Cut-and-Paste Augmentation

This is the area that decided whether the project was feasible. Training the detector needs images of mixed seeds with a bounding box drawn around every single kernel. Producing that by hand is the labelling bottleneck the agriculture surveys warn about: an annotator must click out hundreds of small, similar, overlapping objects per image, and a useful training set runs to thousands of images. The companion tool MultiSeedGen removes that step by synthesising labelled detection data instead of collecting it, and its design draws directly on the synthetic-data literature.

2.4.1 Cut-and-paste compositing

The core technique is cut-and-paste synthesis. Dwibedi et al. [3] showed that pasting cropped object instances onto varied background images produces training data good enough to train competitive detectors, because a detector mostly needs to see an object's local appearance in many contexts, not a globally coherent scene. The key finding in that work is that the paste does not have to look realistic to a human: as long as the local patch statistics are varied, the detector generalises to real images. The main advantage is that the label comes for free. When the generator places a seed cut-out at a known position and scale, it knows the exact bounding box, so every synthetic image is perfectly annotated with no human in the loop.

MultiSeedGen implements this for seeds. It segments individual seed cut-outs from single-seed source photographs, then composites many of them onto a chosen background to build a multi-seed scene, recording a bounding box and class for every placed seed. Because placement is controlled, the generator also writes a per-image multi-label count CSV and COCO annotations that carry provenance for each instance: the source file it came from, and the scale, rotation, shear and perspective applied to it. That provenance is unusual for a synthetic dataset and makes a generated set auditable rather than opaque.

2.4.2 Extracting clean cut-outs

Cut-and-paste only works if the object crops are clean, with the background removed down to the object's true outline. MultiSeedGen offers a cascade of segmentation backends for this. Classical methods (border-colour thresholding, Otsu, OpenCV GrabCut) handle clean and high-contrast backgrounds with no extra dependencies. For harder, feathered edges it falls back to rembg, which runs the U^2-Net salient-object model [19] to separate foreground from background. For

the most difficult cases it can use Segment Anything [20], a promptable segmentation model that produces high-quality masks from a box or point prompt. Both learned backends are import-guarded and optional, so the default install stays light and only pulls in torch when SAM is explicitly enabled. The cut-out quality is gated by a confidence score, and low-scoring masks are dropped before they reach the compositing stage.

2.4.3 Domain randomization and augmentation

A detector trained on synthetic images must transfer to real photographs, and the standard answer is to vary the synthetic data so widely that the real distribution looks like just another sample. Tobin et al. [4] introduced domain randomization for exactly this purpose, randomising textures, lighting and object placement in simulation so a model trained only on synthetic data transfers to the real world without seeing a single real image during training. MultiSeedGen applies the same idea to its scenes. It randomises backgrounds (solid, gradient, noise, texture, and real background crops), per-seed geometry (scale, rotation, flip, shear, perspective jitter), and photometric conditions (gamma, blur, motion blur, sensor noise and vignetting) so the generated set spans far more variation than a hand-collected set would.

These transforms sit within the broader image augmentation literature, surveyed by Shorten and Khoshgoftaar [21], which establishes that geometric and photometric perturbation reduces overfitting and improves generalisation across vision tasks. Augmentation libraries such as Albumentations [22] made these transforms fast and bbox-aware, which matters because moving an image must move its annotations with it. MultiSeedGen handles this carefully: geometric warps that would shift a box at the scene level (scene-wide shear or perspective) are avoided, and the per-seed equivalents are instead applied to each cut-out before placement, so the box is recomputed from the warped alpha mask and stays exact. The pipeline also guards against the failure modes that quietly poison a synthetic detection set: it clips or drops out-of-bounds boxes, limits overlap between placed seeds by an IoU threshold, balances classes during sampling, mixes background-only negative images to suppress false positives, and enforces a deterministic train/validation split where a source seed reserved for validation never appears in any training image. The last point prevents data leakage, where the model in effect sees the validation seeds during training and reports an inflated score.

2.5 MLOps, Model Registries and Experiment Tracking

Training a model is the start; serving and managing many model versions over time is the part that decides whether a system stays trustworthy. As soon as a project trains more than one model, it needs to track what was trained, with which data and parameters, how each version scored, and which version is actually serving traffic. MLflow [23] is a common open-source answer, providing experiment tracking (params, metrics and artifacts per run) and a model registry with

lifecycle stages a model moves through on its way to production.

Seed Bank follows this pattern but implements the registry and serving inside its own domain so that model identity is tied directly to its results. Every model is a row in a `model_artifacts` table carrying its kind, backend, weights location, builder key, per-model configuration, and a lifecycle status that runs `registered` to `staging` to `production` to `archived`, with promotion exposed as an explicit API action. Only staging or production models may serve. Offline evaluation runs a model over a labelled dataset, computes classification metrics, persists them to PostgreSQL, and writes a Markdown report to object storage, so a model can be assessed before it is promoted. At serving time a model resolver selects the model for a given segment: it resolves the production model promoted for that segment, falls back to the global, seed-type-agnostic production model for the kind when a seed type was supplied but no segment model is promoted, and otherwise reports that no model is ready. The resolution is deterministic and stateless, with no weighting; production-model resolution with a global fallback is the whole policy. Every prediction is recorded as an inference row linked to the exact model version that produced it, which gives full traceability from a stored seed label back to its model. This is the operational discipline that most published vision work, which stops at a trained checkpoint, does not address.

2.6 Comparative Analysis

The components above are individually well established. What distinguishes this project is not a new model but how the pieces are assembled into a traceable, operable system, and the decision to remove the data-labelling bottleneck with a purpose-built generator rather than working around it.

Most prior work in seed and grain vision is a single artifact: one classifier or one detector, trained in a notebook against a hand-labelled dataset, evaluated once, and reported. Such a system answers one question (is this seed good, or where are the seeds) but not the combined question the domain actually asks, which is how many seeds are in this photo and how many of them are bad. It carries no record of which model version produced a given result, so a result cannot be reproduced or audited after the fact. It is usually trained on a dataset that was labelled by hand, which caps the dataset size and therefore the accuracy. And it is a model, not a product: there is no API, no client, and no path to swap a model without retraining and redeploying everything.

Seed Bank differs on each of these points. It chains a detector and a per-type classifier into one pipeline so a single photograph yields a located, per-seed good/bad verdict and an aggregate report. Detection and classification are recorded as separate inference rows over the same image, so each stage can be versioned and swapped on its own. Every seed label links back to the exact model version that produced it. The same backend serves a web client, a mobile point-and-shoot flow, and a conveyor-belt batch mode from one inference path. And the data-labelling bottleneck

is solved at the source by MultiSeedGen, which generates perfectly labelled multi-seed detection data with controlled variety and provenance, so the detector can be trained at a scale that hand-labelling could not reach. Table 2.1 summarises the contrast.

Table :2.1 Typical prior work in seed/grain vision versus the Seed Bank platform.

Dimension	**Typical prior work**	**Seed Bank**
Scope	Single model: a detector *or* a classifier	Two-stage detect-then-classify pipeline producing per-seed verdicts and an aggregate report
Quality question answered	"where are the seeds" *or* "is this seed good"	"how many seeds, and how many are bad" over a mixed photo
Deployment form	A notebook or a single trained checkpoint	A served platform with a REST API behind web, mobile and conveyor-belt clients
Model management	One fixed model; replacing it means retraining the system	Registry with a `registered`→`production` lifecycle, hot reload, and per-segment selection
Comparing models	Evaluated once, offline, by hand	Offline evaluation against labelled datasets plus production-model resolution with a global fallback
Traceability	No record of which model produced a result	Every seed label links to the exact model version via an inference row
Crop types	Usually one species, fixed at training	Multiple crop types, each with its own classifier; detections grouped by type
Training data	Hand-labelled, which caps dataset size	Synthetic cut-and-paste generation with free, exact labels and controlled variety
Data integrity	Leakage and label errors are easy to miss	Deterministic leak-free split, IoU-limited overlap, box clipping, and per-instance provenance

The wider point is that the contribution sits at the system level. Each model family here is taken from published work and used as intended; the originality is in binding detection, per-type classification, a versioned registry with promotion-based serving, full traceability, and a synthetic-data generator into one platform that a non-researcher can run, and in attacking the labelling bottleneck head-on rather than living with a small hand-labelled dataset. The chapters that follow describe how that system is built.

Chapter 3

System Analysis

This chapter turns the product idea described in the introduction into a precise statement of what the system must do and the constraints it must satisfy. It identifies who uses Seed Bank and what each of them wants from it, then enumerates the functional and non-functional requirements that the design and implementation are held against. The requirements here are not aspirational: each one corresponds to a capability that exists in the running system, so this chapter doubles as a map between the stakeholders' goals and the endpoints, tables, and workers that satisfy them.

3.1 Stakeholder Analysis

Seed Bank serves two audiences that have almost nothing in common, from a single backend. On one side are the people checking seed quality in a field or a warehouse; on the other are the people who run the machine-learning platform behind the product. The platform's value comes from keeping both groups on the same data: the model that grades a farmer's photo is the same artifact a developer registered, promoted, and measured. Identifying each stakeholder and their goals early kept the requirements honest, because a feature that helps one group must not break the guarantees the other depends on.

The **farmer (end user)** is the primary stakeholder. Their job is to find out whether a batch of seeds is good before planting or buying it. They sign in from the web app or the mobile app, photograph a batch, and expect an instant good/bad report with a seed count and a good-rate, followed by a history they can revisit and, when needed, a read-only link they can share with someone who has no account. Farmers work in Arabic or English, often on a phone, so the interface language and direction matter as much as the inference result.

The **AI developer** is the stakeholder who makes the product accurate over time. They register model weights into the registry, promote a model through its lifecycle from draft to production, and run offline evaluation experiments against labelled datasets to decide whether a new model is actually better than the one in production. They also curate the datasets those experiments run against. Because they need to test a specific model on a real photo, they can override the routed

model at analyze time, which an ordinary user cannot do.

The **administrator** owns the operational levers. They promote models through their lifecycle, manage users and their roles, and read the audit log. Administrators inherit everything the AI developer can do and add the controls that change behaviour for everyone, so the system gates their actions tightly and records them.

Two secondary stakeholders shape requirements without driving the day-to-day flow. **Suppliers** are referenced when a scan is recorded, so that quality results can later be grouped by where a batch came from; the system keeps a catalog of global suppliers plus private ones a developer or admin can add. Finally, the **faculty and evaluators** of the graduation project are stakeholders in the engineering itself: they judge whether the system meets a production-grade bar, which is why the non-functional requirements in Section 3.3 are stated as measurable targets rather than adjectives. Table 3.1 summarises the goals and the main system capabilities each stakeholder relies on.

Table :3.1 Stakeholders, their goals, and the capabilities they depend on.

Stakeholder	Primary goal	Depends on
Farmer / end user	Check seed quality of a batch in the field and review past results	Web/mobile capture, `POST /analyze`, batch polling, share links, EN/AR UI
AI developer	Improve and validate models offline; pick the right model	Model registry, datasets, experiments, per-request model override
Administrator	Control which model serves traffic and manage access	Model promotion, user/role management, audit log
Supplier (secondary)	Be attributable in quality results	Supplier catalog, `supplier_id` on scans
Faculty / evaluators (secondary)	Assess engineering quality against a production bar	Layered architecture, tests, observability

3.2 Functional Requirements

The functional requirements describe what the system does, grouped by the area of the product they belong to. Each requirement maps to a concrete part of the API surface and the data model, so they can be traced into the design and verified against the running service.

3.2.1 Authentication and access

A user can register with an email and password, receive a verification email, and confirm their address before the account is usable. Login issues a short-lived access token and a long-lived

refresh token; the refresh endpoint rotates the refresh token on every use and detects replay of an old token by invalidating the whole chain. Users may also sign in through Google or GitHub using OAuth2 [24]. For programmatic access, a user can create, list, and revoke personal API keys, which are hashed at rest and shown in plaintext exactly once at creation. A guarded one-shot endpoint bootstraps the first administrator.

3.2.2 Analysis and batches

The core action is submitting images for analysis. A user uploads up to sixteen images in a single multipart `POST /analyze` request, optionally pinning a seed type, supplier, model, and geo metadata; this starts one scan batch. Clients then poll `GET /batches/{id}` until the batch reaches a terminal status. Around a batch, the system lets a user list their batches, open one for detail, delete a batch, bulk-delete several, export results as CSV or JSON, download an annotated PNG with the detection boxes drawn on the image, and create or revoke a public read-only share link served at `GET /shared/{token}`.

3.2.3 Analytics and comparison

Beyond a single batch, a user can view aggregated detection and quality metrics over a time window through the analytics endpoint, and compare batches against one another to see how quality differs across scans. The analytics path reads from a separate analytical store so these queries do not load the transactional database.

3.2.4 Model platform

AI developers and administrators operate the machine-learning platform through the API. They register model artifacts with their weights and metadata, list and fetch them, promote a model through its lifecycle, and read per-model performance records. Promotion through the lifecycle determines which production model the resolver selects for a segment, with a global fallback. Developers manage labelled datasets and their items, and create offline-evaluation experiments that run a model over a dataset and compute classification metrics. Switching the production model is a promotion operation in the registry, not a code change.

3.2.5 Administration

Administrators manage the user base: listing users and changing their roles among the three defined roles. Sensitive actions across the system are written to an append-only audit log that an administrator can read.

Table 3.2 assigns an identifier to each functional requirement and records the role allowed to invoke it, following the role-based access control described in Section 3.3.

Table :3.2 Functional requirements with the role permitted to perform each.

ID	Requirement	Role
FR-1	Register with email/password and verify the email address	Public
FR-2	Log in and obtain an access + refresh token pair	Public
FR-3	Refresh the access token with refresh-token rotation and replay detection	Authenticated
FR-4	Sign in with Google or GitHub via OAuth2	Public
FR-5	Create, list, and revoke personal API keys (hashed at rest)	Authenticated
FR-6	Submit up to 16 images in one `POST /analyze` multipart request, starting a batch	End user
FR-7	Poll `GET /batches/{id}` for batch status and results	End user (own)
FR-8	List own batches; open a batch for detail	End user (own)
FR-9	Delete a batch; bulk-delete several batches	End user (own)
FR-10	Export a batch as CSV or JSON	End user (own)
FR-11	Download an annotated PNG of a scanned image	End user (own)
FR-12	Create / revoke a public read-only share link for a batch	End user (own)
FR-13	View a shared batch report without authentication	Public (with token)
FR-14	View aggregated detection and quality analytics over a time window	End user (own)
FR-15	Compare two or more batches	End user (own)
FR-16	List reference seed types and suppliers; create/edit private suppliers	Authenticated
FR-17	Register a model artifact; list/get models	AI developer
FR-18	Promote a model through its lifecycle; read per-model performance	AI developer
FR-19	Override the routed model at analyze time	AI developer
FR-20	Promote or demote a model through its lifecycle so the production model resolves for a segment	Administrator / AI developer
FR-21	Create and manage labelled datasets and their items	AI developer
FR-22	Create / list / get offline-evaluation experiments	AI developer
FR-23	List users and change a user's role	Administrator
FR-24	Read the append-only audit log	Administrator

3.3 Non-Functional Requirements

Non-functional requirements describe how the system must behave while doing the work above. They are the constraints a reviewer would reject the system for missing, so wherever the design fixes a measurable target the requirement states the number rather than an adjective.

Performance. Analysis runs asynchronously from end to end. The `POST /analyze` request

validates and stores the images, commits the batch in the `pending` state, and returns immediately; the detect-then-classify work happens in Celery workers [25], and clients learn the outcome by polling `GET /batches/{id}` roughly every two seconds. A multi-image batch is processed with one task per image, so the images in a batch are inferred in parallel rather than one after another. Nothing in the request path blocks the event loop, because the backend is async throughout [6].

Scalability. The unit of scale is the inference worker. Because each image is an independent Celery task on the `inference` queue, throughput grows by adding worker processes or containers; no part of the design assumes a single worker. Heavy analytical queries are kept off the transactional store by serving them from a separate ClickHouse warehouse [26], and cached or queued state lives in Redis [27].

Security. Access tokens are short-lived JSON Web Tokens [28] with a fifteen-minute time-to-live; refresh tokens live for seven days, rotate on every use, and trigger replay detection that invalidates the chain when an old token is reused. Authorization is role-based across the three roles, enforced on both the API and the clients. Sensitive routes are rate-limited per route via a Redis-backed limiter, API keys are hashed at rest, and every sensitive action is written to an append-only audit log. Social login uses OAuth2 [24]. Errors are returned as RFC 9457 problem details [29] carrying a request identifier, so a client always receives a structured, traceable failure rather than an opaque 500.

Reliability. The batch state machine (`pending` → `running` → `succeeded` / `partial` / `failed`) is driven by compare-and-set updates, so several workers handling the images of one batch cannot corrupt its state: exactly one task owns each transition. When detection has already produced results but classification later crashes, the batch degrades to `partial` and keeps the detections instead of discarding them, and a batch with no routable model fails with a human-readable message rather than a blank error.

Observability. The backend emits the four signals of logs, metrics, traces, and errors, all configured centrally so labels and formats stay consistent. Logs are structured (one JSON object per line outside development) and carry a per-request `request_id` and `path`; Prometheus metrics cover HTTP traffic and domain counters such as inference totals and latency, with the route template used as the `path` label to keep cardinality bounded; traces are exported over OpenTelemetry [30] across the API, database, Redis, outbound HTTP, and Celery; and unhandled exceptions are captured by Sentry. Liveness and readiness probes (`GET /healthz` and `GET /readyz`) let an orchestrator gate traffic, with the readiness probe actively checking Postgres, Redis, MinIO, and ClickHouse.

Usability and internationalisation. The end-user surface is fully bilingual in English and Arabic, including right-to-left layout. The Arabic dictionary is type-checked against the English one so a missing translation is a compile error rather than a silent fallback, and the layout flips using logical CSS properties so dialogs, tables, and navigation render correctly in both directions. The admin and ML pages are intentionally English-only, since they are role-gated to developers and administrators and never reached by farmers.

Maintainability. The backend follows a layered clean architecture [5] in which routers call services, services call repositories, and only repositories touch the ORM [7]. Pydantic schemas validate every boundary and stop the ORM leaking past the service layer, and all configuration comes from one central settings object rather than scattered environment reads. This layering is itself a requirement, because the frontend and backend overhaul recorded in ADR-0001 treats violating it as a defect to fix, not a shortcut to accept. Table 3.3 lists the non-functional requirements with their measurable targets.

Table :3.3 Non-functional requirements and their targets.

ID	Attribute	Requirement / target
NFR-1	Performance	Analysis is async end-to-end; `POST /analyze` returns before inference runs; clients poll at a ∼2 s interval.
NFR-2	Performance	Each image in a batch is a separate task, so images are inferred in parallel; up to 16 images per request.
NFR-3	Scalability	Throughput scales horizontally by adding inference workers; analytics served from a separate ClickHouse store.
NFR-4	Security	JWT access TTL 15 min; refresh TTL 7 days with rotation and replay detection.
NFR-5	Security	RBAC across 3 roles; per-route rate limiting; API keys hashed at rest; append-only audit log.
NFR-6	Security	Errors returned as RFC 9457 problem details with a `request_id`.
NFR-7	Reliability	Compare-and-set batch state machine; classify failure degrades to `partial`, never loses detections.
NFR-8	Observability	Logs, metrics, traces, and errors centralised; `/healthz` and `/readyz` probes.
NFR-9	Usability / i18n	Full EN/AR localisation with RTL across the entire end-user surface.
NFR-10	Maintainability	Layered architecture (routers → services → repositories → ORM); Pydantic at boundaries.

3.4 Use Case Diagrams and Actor Descriptions

The use cases follow directly from the roles. There are three authenticated actors, each inheriting the capabilities of the one below it, plus an unauthenticated actor for the public flows. Figure 3.1 shows the actors and the main use cases.

The **end user (farmer)** is the base actor. Their use cases are signing up and verifying their

email, logging in, submitting a batch of images for analysis, polling that batch until it finishes, browsing their own history, exporting or downloading an annotated result, viewing analytics, comparing batches, and creating a share link. Every one of these acts only on data the user owns.

The **AI developer** extends the end user. On top of every end-user case, the developer manages datasets, registers and promotes models, reads model performance, runs offline experiments, and overrides the routed model when submitting an analysis so they can test a specific model on a real image.

The **administrator** extends the AI developer. Beyond the developer's cases, the administrator promotes models, manages users and their roles, and reads the audit log. These are the cases that change behaviour for other actors, which is why they sit at the top of the inheritance chain.

The **public (unauthenticated) actor** has exactly one use case: opening a shared batch report through its token at `GET /shared/{token}`. This is the only path into the system that requires no account, and it is read-only.

Figure :3.1 Use-case diagram (actors and main use cases).

3.5 Feasibility Study

Technical feasibility. The whole stack runs from a single Docker Compose file [10] on a laptop: the FastAPI backend, Celery workers, Postgres, Redis, MinIO, and ClickHouse come up together with seeded demo data. A GPU is optional; the default inference runtime uses a CPU build of PyTorch, and the GPU runtime is a swap-in target rather than a hard dependency. Every major component is a mature open-source project with a large user base (FastAPI, Celery, PostgreSQL [31], Redis, ClickHouse, React [8]), so there is no bet on an unproven technology and the integration patterns are well documented. The two-tier ML design, where switching the production model is a promotion operation in the registry rather than a code change, keeps the system maintainable as new crop types and models are added.

Economic feasibility. The cost case rests on open-source software and commodity hardware. There are no licensing fees for any component of the stack, and the development and demo environment runs on a single developer machine. A GPU is needed only for training new models or for heavy inference at scale; routine development, the offline-evaluation experiments, and a demo all run on CPU. The object store, warehouse, and tracking server are self-hosted from the same Compose file, so there is no recurring spend on managed cloud services to run or evaluate the system.

Operational feasibility. The product meets each audience where it already is. Farmers reach it through a web app and a mobile app that captures photos with the device camera, both in Arabic or English with right-to-left layout, so the interface does not assume English literacy or a desktop. Developers and administrators reach the same backend through a documented REST API [32] and the role-gated web pages. Because both clients are generated against and validated by the same typed API contract, the operational surface stays consistent across web, mobile, and direct API use, which keeps the system practical to run and to hand over.

Chapter 4

System Design

This chapter describes how Seed Bank is put together: the rules that shape the backend, the runtime topology, the object model behind the inference pipeline, the request and authentication flows, the relational schema, the machine-learning platform that decides which model serves a scan, the synthetic-data tool that feeds the detector, the client interfaces, and the security posture. The design follows the principle that the parts of the system that change for different reasons should be kept apart, so a new inference engine, a new database table, or a new screen can be added without disturbing the rest [5].

4.1 System Architecture

4.1.1 Five architectural pillars

The backend is built on five rules that are enforced by code review rather than by a linter, because they constrain how the layers relate to each other rather than how any single line is written.

The first rule is that the system is asynchronous from end to end. FastAPI runs on an asyncio event loop, the database is reached through async SQLAlchemy 2 over `asyncpg`, and every external client (Redis, MinIO, ClickHouse, outbound HTTP) is async [6]. Nothing in the request path blocks the loop; the one genuinely CPU-bound operation, a PyTorch forward pass, is pushed onto a worker thread with `asyncio.to_thread` so the loop stays responsive.

The second rule is strict layering: `routers` call `services`, which call `repositories`, which talk to the ORM. Routers parse the request, call one service, wrap the result in an envelope, and return; they never import SQLAlchemy. Services hold the business logic and never import FastAPI. Repositories hold exactly one query or one persistence intent each and never embed business rules. Each layer can be tested against the one below it without dragging in the web framework or a live database.

The third rule is that Pydantic validates everything at the boundary. Every request and response is a typed schema, and the ORM model never leaks past the service layer, so the shape of an HTTP payload is decoupled from the shape of a database row. The fourth rule is a sin-

gle `Settings` object, built with pydantic-settings, as the only source of configuration. Values come from environment variables, a `.env` file in development, or one file per field under `/run/secrets` in production; no code reads `os.environ` directly. The fifth rule is model traceability: the chain `seed_detection` → `inference` → `model_artifact` is a hard foreign-key relationship, so any individual seed result can be traced back to the exact model version that produced it.

4.1.2 Context view

At the outermost zoom level the platform is a single boundary that three kinds of actor talk to over HTTPS and JSON. End users (farmers) upload seed images and poll for results. AI developers register and promote models and run offline experiments. Administrators manage users, suppliers, and model promotion. All three hit the same HTTP surface; what differs is the role each one carries and the operations that role is allowed to perform. Outside the boundary sit two OAuth identity providers (Google and GitHub), an optional hosted Roboflow inference backend, and an optional Sentry endpoint for error tracking. Model training happens outside the system entirely: only the trained weights and their metrics are imported, through `scripts/register_model.py` or `POST /models`. Figure 4.1 shows this view.

4.1.3 Container view

One level deeper, the platform is a set of containers declared in a single `compose.yaml`, each one a process with a healthcheck and explicit CPU and memory limits [10]. The `api` container serves every router under `/api/v1`. Two Celery worker containers consume disjoint sets of queues: `worker-inference` runs the detect-then-classify pipeline and offline evaluation, while `worker-cpu` handles data-warehouse writes, experiment bookkeeping, and housekeeping. The split exists so the heavy ML dependencies (`torch`, `torchvision`, around 1.6 GB) load only into the inference worker and never weigh down a routine warehouse write. Four stateful services back the application: PostgreSQL for the OLTP database, Redis for cache, Celery broker, and rate-limit store, MinIO for object storage (images, weights, datasets, experiment reports), and ClickHouse for the analytics warehouse. Experiment metrics persist to PostgreSQL and a Markdown report in MinIO. Every host port is bound to `127.0.0.1`, so nothing is exposed to the local network by default. Figure 4.2 shows the stack.

4.1.4 Component view

Inside the `api` container the layering becomes visible as concrete classes. A request enters through middleware (request-id assignment, structured logging, CORS, the rate limiter, and the problem-details exception handler), reaches a versioned router under `api/v1/`, and is wired to its service through `api/deps.py`, which resolves the current user, checks the

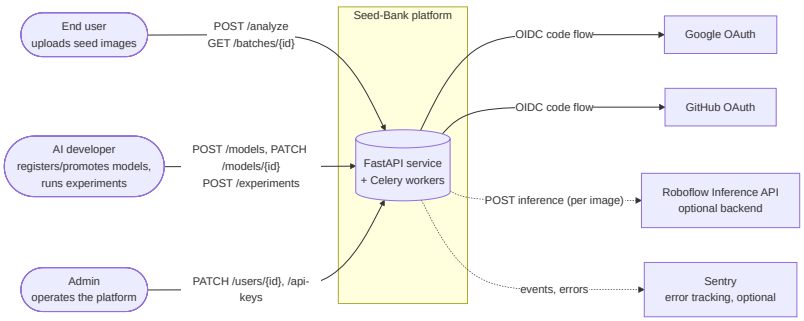

Figure :4.1 System context: the actors and external systems that interact with the Seed Bank boundary.

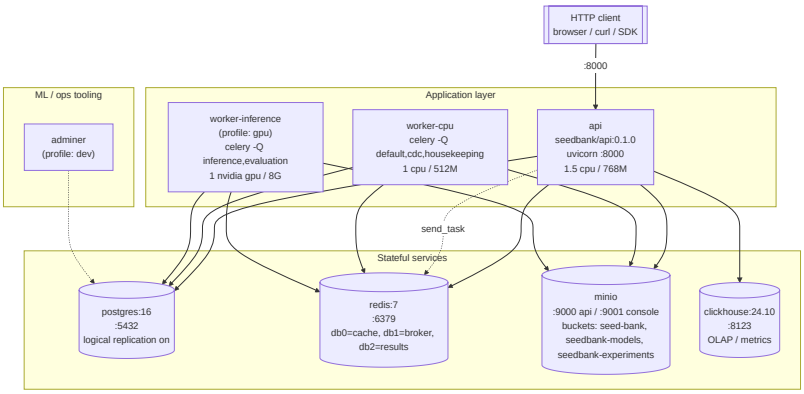

Figure :4.2 Container view of the Compose stack: application processes, stateful services, and ML tooling.

required role, and supplies the database session and repositories. Each router maps to one service: `auth.py` to `AuthService`, `analyze.py` to `AnalysisService`, `batches.py` to `BatchService`, `models.py` to `ModelRegistryService` (with model selection handled by `ModelResolver`), and so on. Services reach the database only through repositories such as `UserRepository`, `ScanBatchRepository`, and `SeedDetectionRepository`, and reach the outside world through infrastructure adapters for storage, cache, OAuth, ML, and analytics. The ORM, a single SQLAlchemy 2 `AsyncSession`, sits at the bottom [7]. Figure 4.3 shows the layering, and Table 4.1 shows how each domain exception that a service may raise is translated into an HTTP status code.

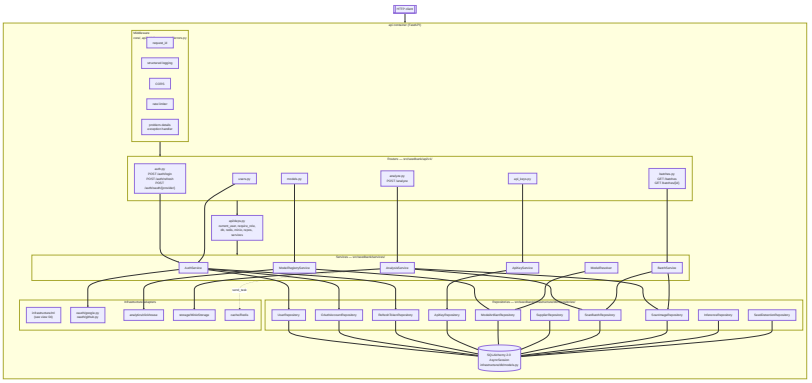

Figure :4.3 API component layering inside the `api` container: middleware, routers, services, repositories, and adapters.

Table :4.1 Mapping from domain exception to HTTP status and problem-details code.

Domain exception	HTTP status	code field
ValidationError	422	validation_error
AuthError	401	auth_error
ForbiddenError	403	forbidden
NotFoundError	404	not_found
ConflictError	409	conflict
RateLimitError	429	rate_limited
ExternalServiceError	502 / 503	external_service_error

The worker container has its own component structure. The Celery app factory routes the `seedbank.analyze_image` task to the `inference` queue with `acks_late` and a prefetch of one, so a worker only acknowledges a task after it succeeds or finally fails, and processes one image at a time. The task is a synchronous entry point that calls `asyncio.run` on the real async work. Each task opens a fresh `AsyncEngine` and session through `worker_session_scope` and disposes it at the end. This is not an optimisation oversight: `asyncpg`'s connection pool binds to the event loop it was created on, and every `asyncio.run` creates a new loop, so reusing the API's process-wide engine would fail with a future attached to a different loop. Within one task the worker runs a fixed sequence: compare-and-set the batch from pending to running, fetch the image bytes from MinIO, resolve the detection model through the model resolver, detect, persist the inference and its detections, commit, resolve a classifier, crop and classify each detection, update the quality column, commit again, then finalise the batch. Figure 4.4 shows these components.

4.2 Class Diagrams

The object model behind the ML pipeline is designed so that the rest of the system never imports PyTorch. The seam that makes this possible is a duck-typed `Protocol`, `InferenceBackend`, with two methods: `detect(image, DetectionConfig)` returns a list of `Detection`, and `classify(crop, ClassificationConfig)` returns a `Classification`. Both return framework-free data-transfer objects. A `Detection` carries a `BoundingBox`, a confidence as a `Decimal`, and a class name; a `Classification` carries a label and a confidence. Because these DTOs hold no tensors, torch never leaks up through the service and router layers, which is why the API image can be built without it.

Three classes satisfy the protocol. `TorchLocalBackend`, the default, loads first-party `.pth` weights from MinIO and runs a local PyTorch forward pass. `YoloBackend` wraps the Ultralytics package for the YOLO family [15], and `RoboflowBackend` calls a hosted Roboflow endpoint over HTTP. Adding a fourth backend means writing one class that matches the protocol and registering it in the pipeline factory; no caller changes. This is the strategy pattern applied to the inference engine [33].

Figure :4.4 Inference worker components: the Celery task, its per-task session scope, and the detect-then-classfiy steps.

Model architectures are kept separate from the engines that run them. A builder is a factory function registered under a `builder_key` that constructs the bare `nn.Module`; weights are loaded into it afterwards. The `ModelBuilder` registry maps a key to its builder, so a new architecture is one new file with a decorator and changes neither the backends nor the router. Builders are append-only by convention: when the math of an architecture changes, the team copies it to a new `-vN` key rather than mutating a builder that a production model already references, because mutating it would silently change what a deployed model computes.

The expensive part, the loaded weights, lives in `ModelManager`, one instance per worker process. It holds a process-wide LRU cache of loaded modules (default four, with separate caches for torch and YOLO). When a model is requested it builds the module through the registry, fetches the weights from MinIO, loads the state dict, moves it to GPU or CPU, and switches it to evaluation mode. Concurrent loads of the same `model_id` are serialised through a per-id `asyncio.Lock` so two tasks never duplicate one GPU load, the least-recently used model is evicted to bound memory, and a model is hot-reloaded when its `model_artifacts.updated_at` advances.

These pieces are coordinated by two services. `ModelResolver` answers the question of which model should run for a given segment, resolving the `production` model promoted for that segment deterministically and with no weighting, and `AnalysisService` drives the request: it persists the batch and its images through `ScanBatchRepository` and `ScanImageRepository`, which expose intents like `cas_status` for the concurrency-safe state transitions. The same service-then-repository layering described for the API holds throughout: a service never issues raw SQL, and a repository never decides policy. Figure 4.5 shows the key classes and their relationships.

4.3 Sequence Diagrams

4.3.1 The analyze flow

The analyze flow is the path the rest of the platform exists to serve. A client sends a multipart `POST /analyze` carrying one or more image files and optional metadata (seed type, supplier, country, GPS, and, for developers, a model override). The ordering inside `AnalysisService` is load-bearing. First it validates every file: the count must be at most sixteen, each must pass size and MIME checks, and each must decode as a real image through Pillow. Then it writes the image bytes to MinIO *before* the database commit, so any committed row always references an object that actually exists. Next it creates the `scan_batch` in the `pending` state along with one `scan_image` per file, writes an audit-log entry, and commits. Only after the commit does it dispatch one Celery task per image onto the inference queue, so a worker can never pick up a task for a batch that is not yet visible in the database. The endpoint returns `202 Accepted` with a `Location` header pointing at the new batch.

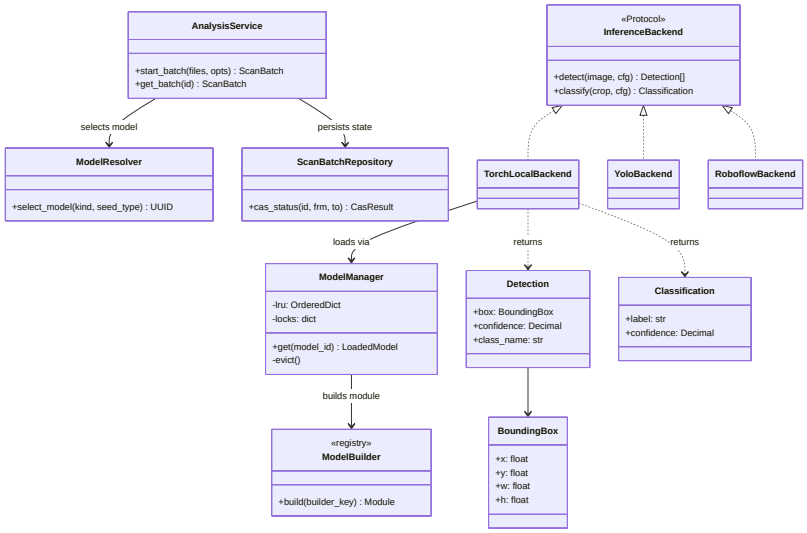

Figure :4.5 Key classes in the ML pipeline: the `InferenceBackend` protocol, its three backends, the builder registry, the model manager, and the services that use them.

Each worker task then runs the per-image pipeline. It compare-and-sets the batch from `pending` to `running`, which only the first task to arrive wins, so several workers processing a multi-image batch cannot corrupt the state. It fetches the image from MinIO, resolves the detection model, runs detection in a thread, and persists one `inference` row and N `seed_detections` rows with normalized bounding boxes. After committing the detection results it resolves a classifier, groups the detections by seed type, crops each detected seed from the source image, classifies it as good or bad, and bulk-updates the quality column. If no classifier is registered for a detected type, those detections are simply left unclassified rather than treated as an error. Finally, when every image in the batch has a detection inference, it compare-and-sets the batch to `succeeded`, `partial`, or `failed`. The client meanwhile polls `GET /batches/{id}` roughly every two seconds until the batch reaches a terminal status, at which point a single eager-loaded query returns the batch with its images, inferences, and detections. Figure 4.6 shows the submission, worker, and polling phases, and Figure 4.7 traces the same pipeline as an activity across the client, API, and worker.

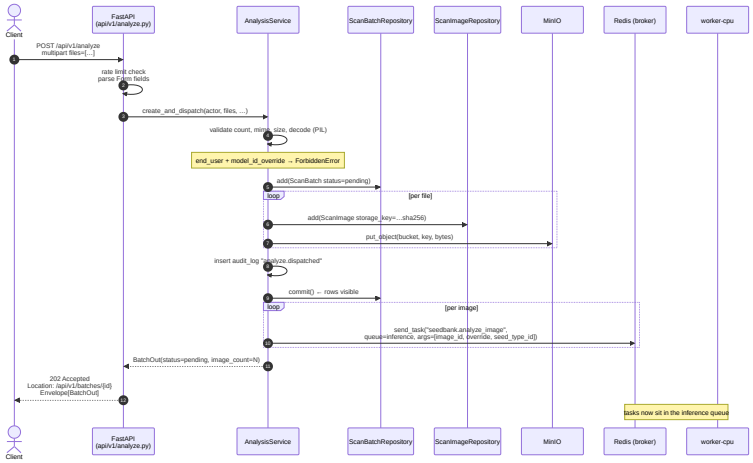

Figure :4.6 Analyze request sequence: submission and dispatch, per-image worker consumption, and client polling.

Figure :4.7 Analyze pipeline activity across client, API, and worker: validate, store, dispatch, detect, classfiy, and the batch state transitions.

Crash safety follows from this ordering. A transient MinIO or Redis fault raises `ExternalServiceError`, which Celery auto-retries up to twice; validation and not-found errors are never retried. Because detection rows are committed before classification runs, a failure during classification degrades the batch to `partial` and keeps the detection data, rather than throwing away good results. If no model can be routed at all, the batch fails with a human-readable message that the clients display, so a user with an unprovisioned deployment sees a clear explanation rather than a blank failure.

4.3.2 Authentication flows

Three authentication flows live behind `/api/v1/auth/`. In local login the service looks up the user by email, verifies the password against the bcrypt hash, and on success inserts a refresh-token row (storing only a SHA-256 hash of the token) and signs a short-lived access JWT carrying the role and scopes [28]. When the access token later expires the client calls `POST /auth/refresh`. The service looks up the refresh token by its hash; if the token is revoked or expired it rejects the request, and if it is valid it revokes the old row, records the new token as its replacement in a rotation chain, and issues a fresh pair. Because each refresh rotates the token and links old to new, presenting a previously used refresh token is detectable as replay and invalidates the chain.

The OAuth flow uses the standard authorization-code exchange [24]. The API redirects the browser to the provider's authorize URL with a `state` value carried across the redirect by the session middleware, receives the callback, exchanges the code for tokens, fetches the user's email and subject, and either finds the linked account or creates a user and links it. It then issues the same token pair as local login. A third path covers headless clients: an API key presented as a bearer credential is recognised by its prefix, short-circuits JWT verification, is hashed and looked up, and yields an authenticated identity with the role and scopes attached to the key. Figure 4.8 shows the login-and-rotation, OAuth, and API-key sequences.

4.4 Database Entity-Relationship Diagram

The OLTP store is a PostgreSQL schema of seventeen tables, managed with Alembic [31]. Every primary key is a UUIDv7 generated in application code, which keeps keys time-ordered and therefore index-friendly while staying globally unique. The tables fall into four groups.

The identity and authentication group has five tables: `users` (email, bcrypt hash, role, verification and active flags), `refresh_tokens` (token hash, expiry, revocation, and the `replaced_by_id` that forms the rotation chain), `oauth_accounts` (provider and subject, encrypted provider tokens), `api_keys` (key hash, prefix, scopes, expiry), and an append-only `audit_log` indexed by actor and time. The reference group has two tables, `seed_types` and `suppliers`, that supply catalog data.

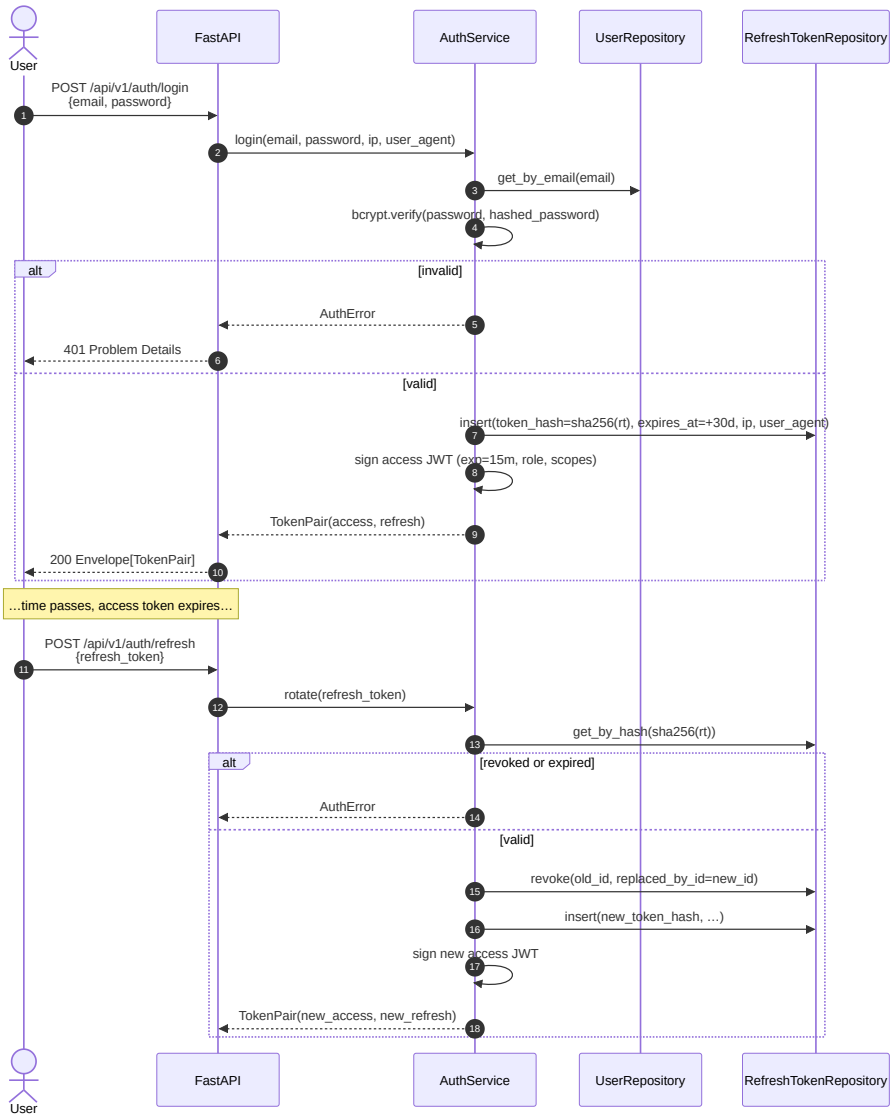

Figure :4.8 Authentication sequences: local login with refresh-token rotation, OAuth code flow, and per-request API-key authentication.

The inference graph is the heart of the schema and the four tables that carry the traceability chain. A `scan_batches` row is created per `POST /analyze` and holds the state machine (`pending` → `running` → `succeeded` / `partial` / `failed`), timing, and optional geo metadata. Each batch groups several `scan_images`, each of which records its MinIO storage key, content hash, and pixel dimensions. An `inferences` row is one model run over one image and records the backend, the `model_id`, the latency, and any error. Each detected seed becomes a `seed_detections` row with its normalized bounding box, confidence, and quality, linked back to its inference. The data fans out as one image to N detections to N quality labels.

The ML-platform group has six tables: `model_artifacts` (registered weights plus metadata and lifecycle status), `model_metrics` (per-model performance records), `datasets` and `dataset_items` (offline-evaluation ground truth), and `experiments` and `experiment_results` (offline-evaluation runs and their computed metrics). Figure 4.9 shows the full schema, and Table 4.2 lists the invariants the design relies on.

Table :4.2 Schema invariants enforced by the database design.

Invariant	What it guarantees
Traceability chain (FK)	Every `seed_detections` row points to one `inferences` row, which points to exactly one `model_artifacts` row.
`uq_inferences_image_model`	One inference row per `(image_id, model_id)`; a re-run with the same model updates rather than duplicates.
`uq_scan_images_batch_storage_key`	Re-uploading the same bytes in the same batch fails fast, because the storage key embeds the SHA-256.
Normalized bounding boxes	Box columns are `NUMERIC(7,6)` in the range $[0, 1]$; pixel coordinates are derived at render time, never stored.
Decimal confidences	`confidence` is `NUMERIC(5,4)`, so arithmetic over the audit trail does not lose precision to float drfit.
Append-only audit log	`audit_log` is never updated or soft-deleted; a promotion records a status transition on `model_artifacts` rather than a destructive rewrite of history.

Storing bounding boxes normalized to $[0, 1]$ as the source of truth has a direct runtime consequence: crops are derived, never stored. The worker multiplies a normalized box by the source image's pixel dimensions at classify time to cut each crop, so the same detection survives image resizing and the database never holds an image. Confidences and box coordinates are also emitted to clients as decimal strings rather than floats, and parsed to numbers only at render time, for the same precision reason.

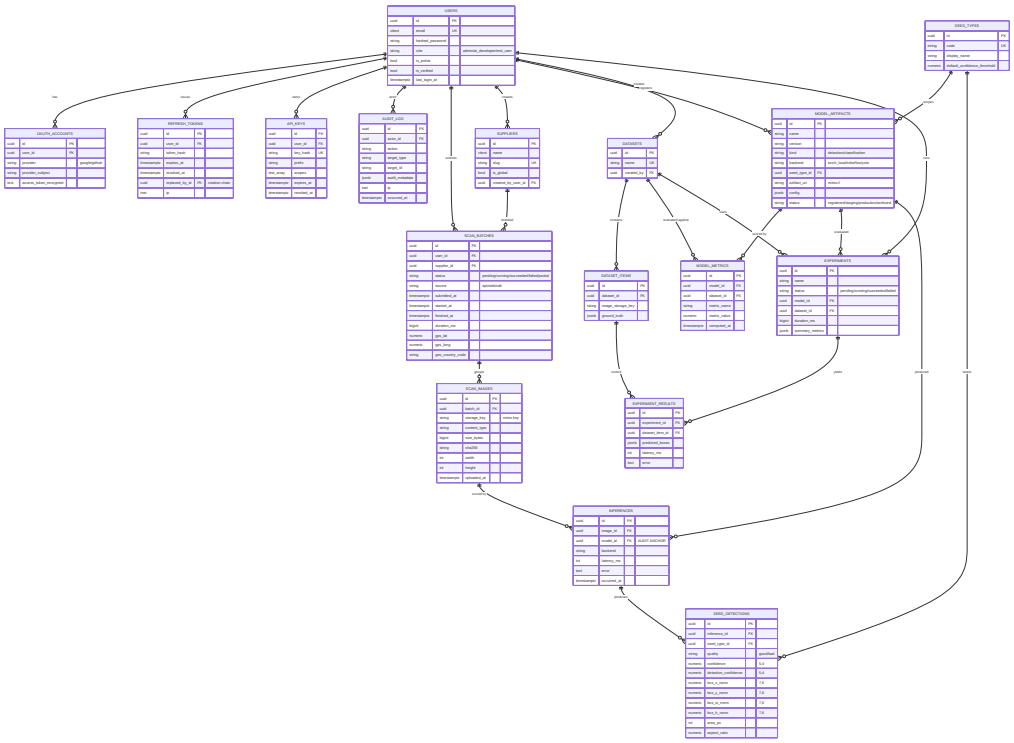

Figure :4.9 Database entity-relationship diagram: identity, reference, the inference traceability graph, and the ML-platform tables.

4.5 ML Platform Design

The ML platform turns a trained `.pth` file into production traffic and lets developers experiment against production without disturbing it. It is organised around three ideas: a registry with a lifecycle, a resolver that selects a model per request, and a manager that loads and caches weights.

4.5.1 Registry and lfiecycle

Weights are uploaded to MinIO and recorded in `model_artifacts`, either through `scripts/register_model.py` or `POST /models`. Each artifact carries its kind (detection or classification), its backend, the MinIO key, a `builder_key`, a free-form `config` JSON of per-model knobs (confidence and IoU thresholds, maximum detections, image size, classification threshold), an optional seed type, and a lifecycle status. The status moves through `registered` → `staging` → `production` → `archived` by an explicit API action, and a model can also be archived directly from `registered` or `staging` when a promotion is abandoned. Only `staging` and `production` models are eligible to serve requests, which is the same condition a developer override must satisfy. The registry governs what actually runs in production, while the metrics from an offline experiment run live in PostgreSQL (the run's `summary_metrics`, the per-item `experiment_results`, and the per-model `model_metrics` rows) together with a Markdown report uploaded to MinIO, surfaced through `GET /models/{id}/performance`. This division of labour, a tracking store for experiment history and a serving registry for production, mirrors the role a tracking tool such as MLflow plays for the former [23].

4.5.2 Model resolution

Given a segment `(kind, seed_type_id)`, the `ModelResolver` decides which model runs. It resolves the `production` model promoted for that exact segment; if none is promoted and a seed type was supplied, it falls back to the global, seed-type-agnostic `production` model for that kind. This fallback makes per-type promotion optional: a deployment can promote a single global model and every scan routes to it, including the mobile point-and-shoot flow, which sends no seed type at all. If nothing is routable the resolver raises `ModelNotReadyError`, which becomes the clear failure message the clients show. Resolution is deterministic and stateless: there is no weighted routing and no per-user bucket. A developer or admin may override the choice with an explicit `model_id` at analyze time, provided it is in `staging` or `production` and matches the requested kind (Figure 4.10).

A developer or administrator can override the resolver with an explicit `model_id` at analyze time. The override must name a `staging` or `production` model of the requested kind, and an end user attempting it is rejected with `ForbiddenError`. This override is the platform's escape

Figure :4.10 Model-resolution activity: the per-request override check, the per-segment production lookup, the global fallback, and the ModelNotReadyError path.

hatch for trying one model on one image without touching production traffic, and the inference row still records the overridden `model_id`, so the result remains traceable. Figure 4.11 shows the lifecycle, the resolver's decision tree, and the override path.

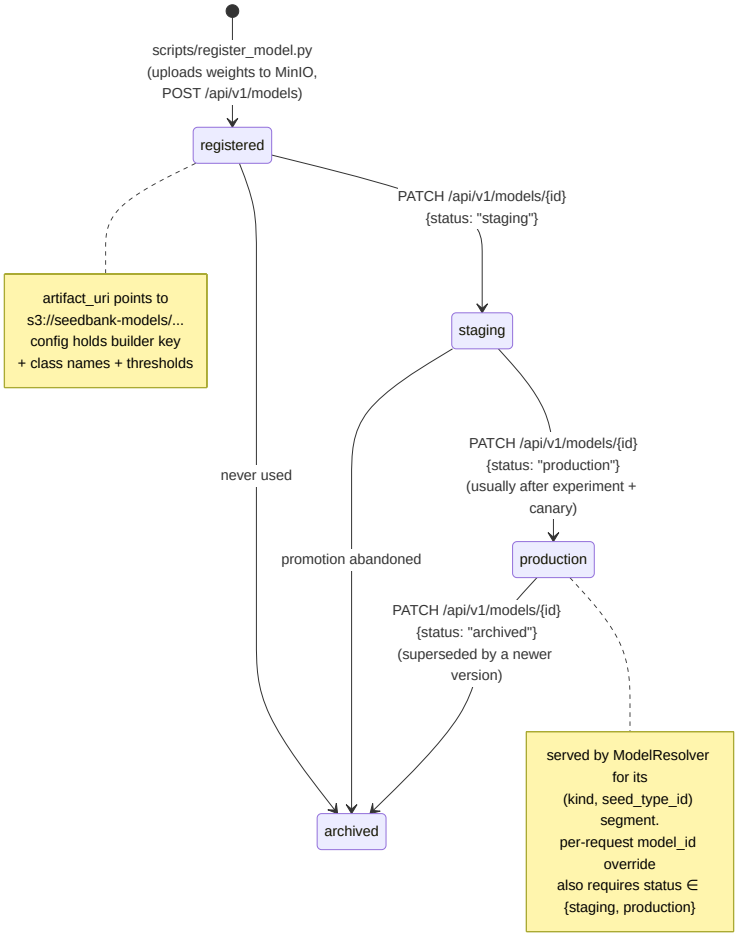

Figure :4.11 Model lfiecycle, production-model resolution with a global fallback, and the per-request developer override.

4.5.3 Model manager

The router decides which model; the manager loads it. Each worker process holds one `ModelManager` with a process-wide LRU cache, a per-id load lock, and a hot-reload check against `updated_at`, as described in the class-diagram section. The two wired architectures show why the builder and backend seams matter. Detection uses a Faster R-CNN with a ResNet-50 backbone and a Feature Pyramid Network and a three-class head over background, coffee, and maize [11]–[13]; the backend filters by the configured confidence threshold, normalizes the pixel boxes, and caps the count. Classification uses a ResNet-18 backbone with a Convolutional Block Attention Module inserted after the last residual stage, then hybrid average-and-max pooling into a single-logit head [18]. There is one classifier per crop type,

which is why detections are grouped by seed type before classification. Because a detector and a classifier are recorded as separate inference rows over the same image, each can be versioned and swapped independently.

4.6 MultiSeedGen Design

Training a seed detector needs many photographs of cluttered, overlapping seeds with a correct bounding box on every seed, which are slow to capture and tedious to annotate by hand. MultiSeedGen is the companion tool that manufactures this data: it takes photographs of single seeds and composites them into annotated multi-seed scenes whose labels are exact by construction, an instance of cut-and-paste synthesis [3]. The whole pipeline is driven by one flat Pydantic configuration object, and its output is byte-identical for a fixed configuration and seed, which lets a generated dataset be reproduced exactly.

The pipeline runs in stages, shown in Figure 4.12. It begins with a segmentation cascade that cuts each seed out of its source photo: a classical colour-and-brightness method (border colour, then Otsu thresholding, then GrabCut) is tried first, and a learned matting model is the fallback, either U2-Net through rembg [19] or Segment Anything [20]. A confidence gate decides whether the cut-out is clean enough, and every result is stored in a content-hashed disk cache so a seed is segmented once and reused across runs. The clean cut-outs form a seed pool.

Scene composition then paints seeds onto a canvas. It samples a number of seeds per scene, places each one while rejecting any placement whose box overlaps an existing box beyond a configured IoU limit and that fails a visibility check, applies label-safe geometric and photometric augmentation (scale, rotation, shear, and HSV jitter chosen so they do not move a label box), composites onto a background that is solid, gradient, noise, texture, or a real inpainted photo, and adds soft directional shadows. Because the tool knows exactly where it placed each seed, label derivation is occlusion-aware: it produces axis-aligned bounding boxes and polygons that account for seeds partly hidden behind others, and records provenance for each box back to its source seed. The composed scene then passes through camera-matched degradation (blur, noise, JPEG compression, and gamma or vignette) so the synthetic images resemble real phone photos, a form of domain randomization that helps a model trained on them transfer to real input [4], [21].

Export writes the dataset in the formats a detector trainer expects: YOLO boxes, YOLO-seg polygons, COCO annotations [17], and a `data.yaml` plus a run manifest. The final stage is a leakage-safe split: a source seed reserved for validation never appears in any training image, enforced per class and deterministically, so the validation score is not inflated by the same seed appearing on both sides of the split. The resulting dataset is what trains the Seed Bank detection model.

Figure :4.12 MultiSeedGen synthetic-data pipeline: segmentation cascade, scene composition, occlusion-aware labels, degradation, export, and leakage-safe split.

4.7 User Interface Design (Wireframes and Mockups)

Seed Bank ships two clients that share a contract and a visual language. The web app is a React 18 single-page app served by Vite, and the mobile app is built with Expo and React Native [8], [9]. Both speak the same `/api/v1` paths, the same `{data}` response envelopes, and the same bearer authentication with a single transparent refresh on a 401. Both wear an agricultural-green theme, a deep leaf-green primary with warm amber accents over a warm off-white field in light mode and a soil-night palette in dark mode, defined as HSL CSS variables so light and dark swap without a rebuild. Both are bilingual: English and Arabic, with Arabic flipping the entire layout to right-to-left.

4.7.1 Web app

The web app opens on a sign-in screen with a field-pattern backdrop and theme and language toggles available before authentication, shown in Figure 4.13. After sign-in the dashboard (Figure 4.14) presents a strip of key figures (scans, images, success rate, and a fourteen-day sparkline), quick actions, and recent scans. The core journey runs through the analyze page (Figure 4.15), where a user drags in or picks photos, optionally adds metadata, and submits, which starts a batch and navigates to its detail. The batch detail page (Figure 4.16) polls until the batch is terminal, then shows an insights panel (a quality donut, a confidence histogram, and headline stats), a per-seed-type breakdown, and per-image cards with an interactive bounding-box overlay that can filter by quality, threshold by confidence, and toggle labels. From here a user can export the results as CSV or JSON, or create a public read-only share link.

[FIGURE PLACEHOLDER]

figures/web-login.png

Web sign-in, with theme and language toggles available before authentication.

Figure :4.13 Web sign-in, with theme and language toggles available before authentication.

[FIGURE PLACEHOLDER]

figures/web-dashboard.png

Web dashboard, showing the key-figure strip, quick actions, and recent scans.

Figure :4.14 Web dashboard, showing the key-figure strip, quick actions, and recent scans.

Right-to-left support is achieved by converting directional Tailwind utilities to logical properties in the shared primitives and pages, with direction-aware chevrons and a drawer that opens from the correct edge. Numbers stay in Latin digits in Arabic for cross-script consistency, and

[FIGURE PLACEHOLDER]

figures/web-analyze.png

Web analyze page, where photos and optional metadata are submitted to start a batch.

Figure :4.15 Web analyze page, where photos and optional metadata are submitted to start a batch.

[FIGURE PLACEHOLDER]

figures/web-batch-detail.png

Web batch detail with the interactive bounding-box overlay and the insights panel.

Figure :4.16 Web batch detail with the interactive bounding-box overlay and the insights panel.

dates are formatted with the active locale. The entire end-user surface is translated; the role-gated admin and ML pages are deliberately left in English, since farmers never reach them.

4.7.2 Mobile app

The mobile app is built for farmers in the field, and its headline screen is the camera (Figure 4.17): a live preview with a centre framing guide, a shutter that captures multiple shots into a review strip, and a "Check seeds" button that uploads the captured photos as one batch. The result screen (Figure 4.18) then polls the batch until it is terminal and shows the good-rate, the seed counts, and the confidence. The same capture-and-submit design extends to a production line: up to sixteen frames of seeds passing under a fixed camera accumulate in the review strip and are submitted as one batch, then graded in parallel by the inference workers, so an operator gets a report on the whole pass within seconds without any belt-specific code.

[FIGURE PLACEHOLDER]

figures/mobile-camera.png

Mobile camera capture, with a live preview, framing guide, and multi-shot review strip.

Figure :4.17 Mobile camera capture, with a live preview, framing guide, and multi-shot review strip.

4.7.3 MultiSeedGen interface

MultiSeedGen has its own web interface, shown in Figure 4.19: a React single-page app served by a thin FastAPI wrapper that drives the same command-line pipeline as a subprocess and

[FIGURE PLACEHOLDER]

figures/mobile-result.png

Mobile result screen, polling the batch and showing good-rate, counts, and confidence.

Figure :4.18 Mobile result screen, polling the batch and showing good-rate, counts, and confidence.

streams its log over a websocket. The interface is organised into four tabs, for running a generation, tuning the segmentation, browsing generated datasets, and editing the configuration. The configuration form is built from the same Pydantic model that defines the CLI, so the form fields, their choices, and their defaults can never drift from what the pipeline actually accepts.

[FIGURE PLACEHOLDER]

figures/msg-ui.png

MultiSeedGen web UI, with tabs for running, segmentation tuning, dataset browsing, and configuration.

Figure :4.19 MultiSeedGen web UI, with tabs for running, segmentation tuning, dataset browsing, and configuration.

4.8 Security Design Considerations

Security is built into the layers rather than bolted on at the edge, and the design choices below cover authentication, authorization, transport, and error handling.

Local authentication stores passwords only as bcrypt hashes; a plaintext password is never written anywhere. Sessions use two JSON Web Tokens, a short-lived access token (fifteen minutes) and a long-lived refresh token (days), and the refresh token rotates on every use [28]. Only the SHA-256 hash of a refresh token is stored, each rotation links the old token to its replacement, and presenting a previously used token is treated as replay and invalidates the whole chain. Social login is offered through Google and GitHub over the OAuth authorization-code flow, with the `state` parameter carried across the redirect to guard against cross-site request forgery on the callback [24]. API keys give headless clients a credential that is hashed at rest, carries a recognisable prefix, and is shown in plaintext exactly once at creation; like refresh tokens, the stored form is a hash, so a database leak does not expose a usable key.

Authorization is role-based across three roles. An `end_user` can analyze and see only their own history; an `ai_developer` adds model, dataset, and experiment access and the per-request model override; and an `admin` adds user management. Roles are checked at the API through a `require_role` dependency and again in the clients' routing, and the model override is the most heavily tested authorization boundary in the system because it is the one place a non-admin role

gains a privileged capability. Each route also carries its own rate limit, backed by Redis through slowapi, with tighter caps on login, registration, refresh, and analyze than on the general default.

Every sensitive action is written to the append-only audit log, which records the actor, the action, the target, and the time, and is never updated or deleted. Secrets are read from environment variables in development but from one file per field under `/run/secrets` in production, and a pre-commit secret scanner keeps credentials out of version control. Errors are returned as RFC 9457 problem-details JSON with a correlating request id and never as HTML pages, so an error surface leaks no stack traces [29]. The deployment binds every host port to the loopback interface, restricts cross-origin requests to a configured allow-list, and enforces a trusted-hosts list, so the attack surface exposed by default is small.

Chapter 5

Implementation

The previous chapter described what the system should do and how its pieces fit together. This chapter is about how those pieces were actually built: the development environment that lets one developer run the whole platform on a laptop, the implementation choices that turned the design into working code, and a selection of screenshots and generated artefacts that show the result. Where a number is recorded in the project's own configuration or documentation it is quoted directly; where no measurement exists, the mechanism is described instead of inventing a figure.

5.1 Implementation Environment and Setup

Seed Bank runs from a single Docker Compose file [10]. The goal was that a fresh checkout reaches a working stack in a handful of commands, without anyone installing Postgres, Redis, MinIO, ClickHouse, or a Python toolchain on the host. Compose profiles keep the default footprint small: the plain `make up` brings up only what the application needs to serve requests (the API, the two Celery workers, Postgres, Redis, MinIO, and ClickHouse), and the heavier extras sit behind named profiles. The `dev` profile adds Adminer and `ch-ui` for inspecting the two databases in a browser; the `obs` profile adds Prometheus and Grafana; the `frontend` profile adds the built React app served by nginx. A developer who only wants the datastores runs `make up-infra`, which skips every image build and is up in about ten seconds. The full lean stack fits in roughly 3.5 GB of RAM at idle. Figure 5.1 shows the deployment topology and how the containers connect over the internal bridge network.

The host ports are bound to `127.0.0.1` so nothing is exposed to the LAN by default. A shared environment anchor feeds the same datastore DSNs and credentials to the API and both workers, so there is one place to change a connection string. When two stacks collide on the same dev machine, a gitignored `compose.override.yaml` shifts the ports without touching the checked-in file. Each service declares a healthcheck and a CPU/memory cap, and `depends_on` with `condition: service_healthy` gives a correct boot order so the API waits for its dependencies before it starts accepting traffic.

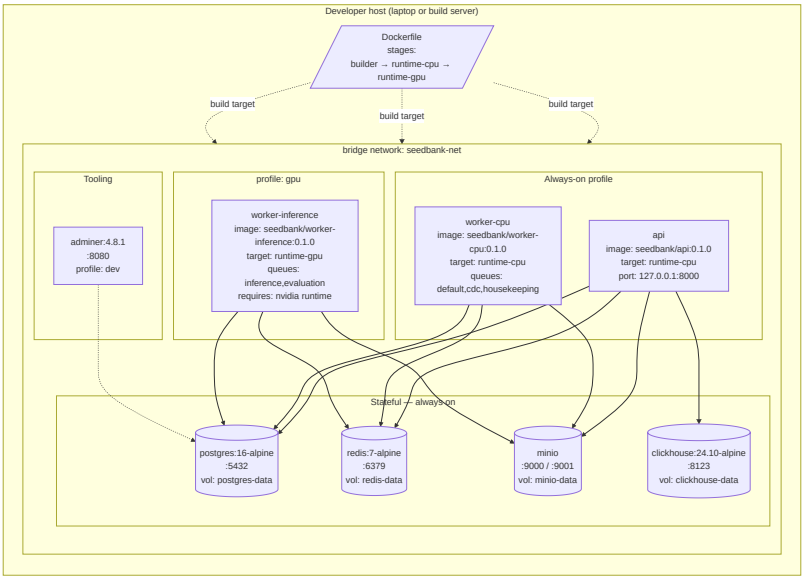

Figure :5.1 Deployment topology: the lean Compose stack, the profile-gated extras, and the production overlay.

5.1.1 Container images

One multi-stage `Dockerfile` builds every first-party image. Four builder stages resolve dependencies with `uv` and feed five runnable runtimes, and each runtime carries only what its job needs. The `runtime-cpu` target backs both the API and the `worker-cpu` container; it has the web, database, and storage dependencies but no `torch` at all. The `runtime-inference-cpu` target adds slim CPU `torch`, `torchvision`, and a headless OpenCV (about 1.6 GB) so the `torch_local` backend can run; this is what the `worker-inference` container uses in development. `runtime-inference-cpu-full` layers Ultralytics (YOLO) and the Roboflow inference SDK on top for the optional backends [15], and `runtime-gpu` swaps in a CUDA 12.4 base with GPU `torch` and a device reservation. Keeping `torch` out of the API image is a deliberate size and start-up decision: a routine analytics dual-write never pays a ~1.6 GB import, because the process that does that work has no `torch` installed.

5.1.2 Bringing the stack up and registering a model

The Makefile is the single entry point for running and inspecting the system. A first run on a clean checkout is `make env` (creates `.env` from the example), `make up`, `make migrate` (Alembic to head), and `make seed`. The seed step runs the ClickHouse schema first, then loads the catalog of seed types and suppliers, registers the bundled models, and creates demo users. Until at least one detection model and one classification model are registered and promoted to `production`, the platform cannot infer; that is by design, and the failure it produces is a clear message rather than a blank error. Model weights are not stored in git. MinIO is the source of truth, and a model is added either through `scripts/register_model.py` or `POST /models`, then promoted through its lifecycle. The `/readyz` probe reports each backing component's status independently, so a developer can tell at a glance whether the stack is healthy or, say, only ClickHouse is still warming up.

The production overlay is a single `compose.prod.yaml` layered on the dev file. It sets deltas, not new service definitions: file-based secrets read from `/run/secrets` instead of environment variables, dropped external ports (only the API and Grafana stay reachable), restart policies, log rotation, and the GPU inference worker on by default. `make up-prod` runs a `secrets-check` first that fails the bring-up if any expected secret file is missing or has permissions other than `0400`.

5.2 Key Implementation Details

5.2.1 Two-Stage Detection and Classfiication Pipeline

Checking a photo of seeds is two separate machine-learning problems, and the implementation treats them as two models run in sequence. The first stage is detection: an object detector

scans the whole image and returns a bounding box, a confidence, and a class for every seed it finds, so one photo becomes N located seeds. The detector in use is a Faster R-CNN [11] built from torchvision's `fasterrcnn_resnet50_fpn`, a ResNet-50 [13] backbone with a Feature Pyramid Network [12] and a three-class head, `[background, coffee, maize]`. The backend takes the model's raw `boxes`, `scores`, and `labels`, filters them by the per-model `confidence_threshold`, normalizes the pixel boxes to the $[0, 1]$ range, and caps the result at `max_detections`. The class id is mapped back to a crop name through `{0: background, 1: coffee, 2: maize}`.

The second stage is classification. For each detection, the worker crops that bounding box out of the original image and feeds the crop to a binary classifier that returns `good` or `bad` with a confidence. The classifier is an ImageNet-pretrained [16] ResNet-18 with a CBAM attention module [18] inserted after `layer4`. Instead of a single global pool, it concatenates a global-average and a global-max pool ($512 + 512$) into a 1024-feature vector that feeds a single-logit `fc` head. At inference the crop is resized to 224×224, normalized with the ImageNet statistics, run forward, and passed through a sigmoid; the label is `good` when the score is at or above the threshold and `bad` otherwise, with the confidence being the score for `good` and its complement for `bad`. There is one classifier per crop type, a coffee classifier and a maize classifier, which is why detections are grouped by seed type before the classify stage and each group is routed to the matching model.

Two implementation decisions in this pipeline are worth naming. First, detection and classification are recorded as separate `inferences` rows over the same image. That separation is what lets a detector and a classifier be versioned and swapped independently instead of being welded into one opaque step. Second, the crops are never stored. The normalized bounding box is the source of truth, and the worker re-derives each crop by multiplying the box by the source image's pixel dimensions at classify time, so there is no second copy of the image data to keep consistent.

5.2.2 Inference Backends and the Model Manager

The engine that actually runs a model is a pluggable backend. Every backend satisfies one duck-typed `Protocol` with two methods, `detect(image, DetectionConfig)` and `classify(crop, ClassificationConfig)`, and both return framework-free data-transfer objects (`BoundingBox`, `Detection`, `Classification`) so that `torch` types never leak up into the service or API layers. Three backends are registered. The default, `torch_local`, builds the module from the architecture registry, loads its state dict from MinIO weights with `weights_only=True` and `strict=False`, and runs the forward pass in a worker thread via `asyncio.to_thread` so the event loop stays responsive; it uses CUDA when a GPU is present. The `yolo` backend loads `.pt` weights into an Ultralytics model [15], and the `roboflow` backend calls a hosted inference service over the network with no local weights. Adding a backend is writing

a class that matches the protocol and registering it in the factory. The heavy imports for these engines live inside the method bodies, not at module top level, which is the concrete reason the API image can ship without `torch`.

Loading weights is expensive, so each worker process owns one `ModelManager` that holds a process-wide LRU cache of loaded modules, with separate caches for the torch and YOLO engines and a default ceiling of four models. Loading a model means building the module from the registry, fetching the weights from MinIO, loading the state dict, moving it to CUDA or CPU, and calling `eval()`. Concurrent loads of the same `model_id` are serialized through a per-id `asyncio.Lock`, so two tasks that need the same model at once never duplicate a GPU load; the cache evicts least-recently- used entries to bound GPU memory. The manager also hot-reloads a model when its `model_artifacts.updated_at` timestamp advances, which lets an operator push new weights without restarting a worker. Pipelines themselves are cheap and created per request; only the weights are cached.

5.2.3 Concurrency-Safe Batch State Machine

A single `POST /analyze` can carry up to sixteen images, and each image becomes its own Celery task on the `inference` queue. Several workers can therefore be touching the same `scan_batch` row at the same time, which is exactly the situation where naive status updates corrupt state. The batch state machine avoids that with compare-and-set transitions: the move from `pending` to `running` succeeds for only the first task that arrives (which also stamps `started_at`), and the move from `running` to a terminal status succeeds for only the task that observes every image as detected. Exactly one task owns each transition, so no amount of concurrency can leave the batch in a half-written state. Figure 5.2 shows the full state machine and its transitions.

The terminal states encode a deliberate failure policy. A clean run where every image is detected and classified ends `succeeded`. If no routable model exists, the batch ends `failed` with a human-readable `error_message` that tells the user an administrator must register and promote a model. The case worth calling out is a classify failure that happens after detection has already been persisted: rather than discard good detection results, the batch degrades to `partial`. The detection data is kept and shown, and only the quality labels are missing. Transient errors from MinIO or Redis are handled separately; they raise an `ExternalServiceError` that Celery auto-retries up to twice before the batch is failed.

5.2.4 Model Resolution

Which model serves a given request is decided by the `ModelResolver`, keyed on a segment of `(kind, seed_type_id)`. It resolves the `production` model promoted for that segment; if none exists and a seed type was supplied, it falls back to the global, seed-type-agnostic `production` model for that kind. That fallback chain makes per-type promotion optional: a deployment can promote a single global model and every scan routes to it, including the

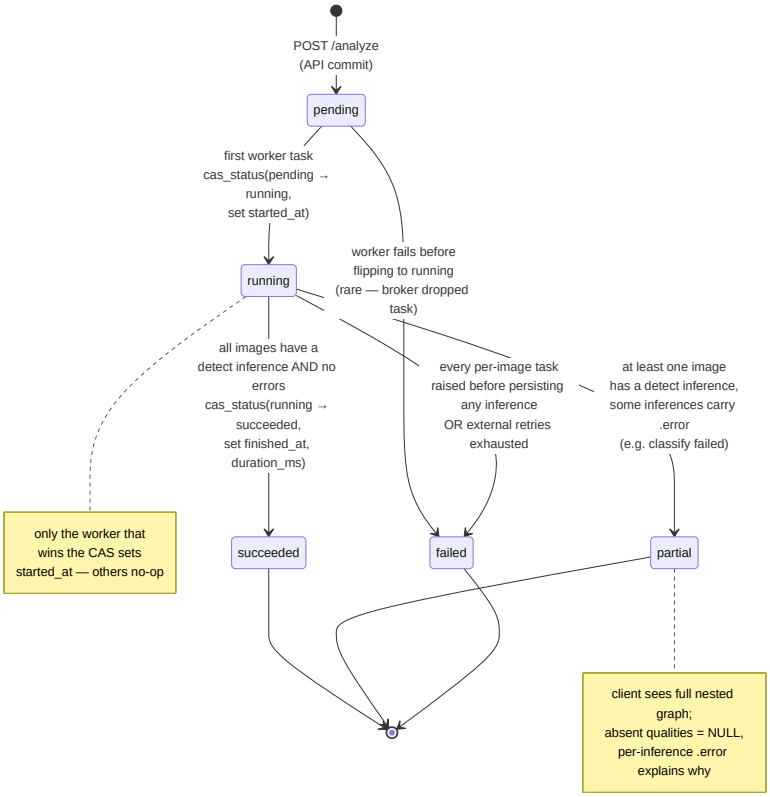

Figure :5.2 The scan-batch state machine. Each transition is a compare-and-set update so concurrent workers cannot corrupt the batch status.

mobile point-and-shoot flow, which sends no seed type at all. If nothing is routable the resolver raises `ModelNotReadyError`, surfaced to the clients as a clear failure rather than a blank error. Resolution is deterministic and stateless. A developer or admin can override the model with an explicit `model_id` at analyze time, provided it is in `staging` or `production` and matches the requested kind.

5.2.5 Authentication: Token Rotation and Replay Detection

Authentication issues a short-lived JWT [28] access token alongside a long-lived refresh token. The refresh token rotates on every use: presenting a valid refresh token returns a new one and invalidates the old. The safety property is replay detection. If a refresh token that has already been rotated out is presented again, the whole token chain is invalidated, so a leaked token that an attacker tries to reuse takes the legitimate session down with it instead of granting silent access. Clients store both tokens and transparently refresh once on a `401` before retrying the original request, so rotation is invisible in normal use. The same security surface includes OAuth2 [24] social login through Google and GitHub, hashed-at-rest API keys shown in plaintext exactly once at creation, per-route rate limiting backed by Redis, and an append-only audit log of sensitive actions.

5.2.6 Data Model Precision and Identfiiers

The OLTP schema is eighteen tables on Postgres [31], reached only through repository classes so that the rest of the code never touches the ORM directly [7]. Two implementation details matter for data quality. First, every primary key is a UUIDv7. These are time-ordered, so they sort and index like a sequential key without the contention of a shared counter and without leaking a monotonic row count. Second, the inference graph uses exact numeric types instead of floats: `confidence` is `NUMERIC(5,4)` and the normalized bounding-box columns are `NUMERIC(7,6)`. The worker rounds its values and passes `Decimal` objects, so SQLAlchemy never coerces a binary float into a column of a different scale and the stored coordinates are exactly what the model produced. User-visible aggregates use a soft delete: a batch the user removes from their history is marked deleted rather than physically dropped, which keeps the traceability chain intact.

5.2.7 App-Level Data Warehouse Dual-Writes

Analytics queries run against a ClickHouse [26] star schema rather than the transactional Postgres, to keep heavy aggregation off the serving database. The schema is populated by an app-level dual-write, not change data capture. After a batch, inference, or detection is committed in Postgres, the application dispatches a Celery [25] task that writes the corresponding rows to the warehouse's dimension and fact tables. The fact tables use `ReplacingMergeTree` and are

partitioned by month, so a repeated write of the same key is idempotent at merge time and a re-dispatched task does not double-count. The dual-write is best-effort and runs after the Postgres commit, which means the OLTP transaction is never held hostage to ClickHouse availability: if the warehouse is down, the source-of-truth write still succeeds, the dispatch is recorded as failed in `seedbank_dwh_dispatch_total`, and the data can be recovered later by a backfill from Postgres. The analytics router and the web Analytics page read only from this warehouse. Postgres is configured with `wal_level=logical` and replication slots so a true CDC pipeline can replace the dual-write later without a schema change, but today the app-level path is what runs.

5.2.8 Observability

The backend emits four signals, all wired in one place so labels and formats stay consistent. Logging uses `structlog`: pretty colourised console output in development and one JSON object per line everywhere else, for a log aggregator to ingest. Every line is enriched with context variables bound per request, so `request_id` and `path` attach automatically to every log emitted while a request is in flight. The rule the code follows is to log through `structlog.get_logger` and never the stdlib logger, because the context variables only attach on the former.

Metrics are Prometheus counters, histograms, and gauges owned by a dedicated `CollectorRegistry` rather than the global default, which avoids duplicate time-series when test fixtures rebuild the app. The discipline that keeps the metrics from overwhelming Prometheus is cardinality control: the `path` label is always the Starlette route template, `/api/v1/batches/{id}`, resolved by re-walking the router after the handler runs, never the raw URL, so per-UUID requests do not explode into millions of series. Status codes are bucketed into `2xx`/`3xx`/`4xx`/`5xx` classes rather than kept per code, the inflight gauge is labelled by method only, and `/metrics` is excluded from its own histograms. Prometheus applies a second guard, a relabel rule that drops any series whose `path` still looks like a UUID, as a fail-safe if templating ever regresses.

Tracing is OpenTelemetry [30] with an OTLP exporter and auto-instrumentation across every IO boundary the service crosses: FastAPI requests, SQLAlchemy and asyncpg, Redis, outbound httpx, and Celery tasks, so a single trace can follow a request from the API, through the queue, into a worker's database calls. It is opt-in and idempotent, a complete no-op unless `OTEL_EXPORTER_OTLP_ENDPOINT` is set, so the dev stack never dials a dead collector; Celery re-initialises it per forked worker because forking after init breaks the gRPC channel. Errors go to Sentry, also a no-op unless `SENTRY_DSN` is set, with `send_default_pii` off so no personal data is shipped.

5.2.9 Bilingual RTL Engine

Both clients ship in English and Arabic with full right-to-left support, built on a dependency-free, fully-typed i18n layer rather than a third-party framework. The English dictionary is the source of truth: `keyof typeof en` becomes the `TranslationKey` type, and the Arabic dictionary is typed `Record<TranslationKey, string>`, so a missing or extra Arabic key is a compile-time error and the UI can never half-fall-back to English. The hook exposes `t` for interpolated lookups and `tn` for pluralization, which selects the correct CLDR plural form (English one/other; Arabic uses the full zero/one/two/few/many/other set). On the web [8], RTL is achieved by writing the shared UI primitives in logical Tailwind properties (`ms-`, `me-`, `ps-`, `pe-`, `start-`, `end-`) so the same component mirrors itself when the document direction flips, with direction-aware chevrons and drawers that open from the correct edge. On mobile [9], React Native only mirrors the layout on the next launch, so switching to or from Arabic calls `I18nManager.forceRTL` and then triggers an `expo-updates` reload in production builds; plural categories are computed by hand because Hermes ships only a partial `Intl`. Numbers stay in Latin digits in Arabic for cross-script consistency, and dates are formatted with the active locale. The entire end-user surface is translated; the developer- and admin-only ML pages stay English on purpose, since farmers never reach them.

5.2.10 MultiSeedGen Synthetic-Data Pipeline

Training a detector needs labelled multi-seed images, which are far harder to collect than the single-seed photos that exist in the source datasets. MultiSeedGen closes that gap by cutting individual seeds out of source photos, compositing many of them onto realistic backgrounds, and exporting a labelled detection dataset. The whole tool is built around one guarantee: for a fixed `(config, seed)` the output is byte-identical regardless of how many worker processes run it. That determinism is the backbone of the pipeline and shapes every design choice below.

Segmentation, cutting one seed cleanly out of its source, runs as a cascade. The default `auto` method tries a border-colour distance threshold, then Otsu, then OpenCV GrabCut, escalating only when the cheaper method scores poorly, with `rembg` (a U^2-Net ONNX model [19]) and an optional Segment Anything backend [20] as fallbacks for hard, feathered edges. A confidence skip-gate drops any cut-out whose mask scores below a threshold (default `0.55` for `auto`), so a bad segmentation is discarded rather than baked into the labels. Every cut-out is cached on disk under a content hash plus a segmentation-version and a frozen parameter signature, so the first pass over a source library is the only expensive one and later runs are near-instant; the resolved device is folded into the cache key because GPU and CPU masks can differ very slightly.

Scene composition draws on a per-(split, scene-index) seed sequence so a scene's pixels never depend on worker count or scheduling. Seeds are placed with an IoU limit that rejects overlapping boxes, and the exported box or polygon is recomputed from each seed's warped alpha so it stays exact even under per-seed shear or perspective; scene-wide warps that would

move a label are avoided on purpose [3]. Augmentation is label-safe, applied to the cut-out before placement, and the composite is finished with camera-matched degradation, sensor noise, JPEG compression, mild blur, and gamma shift, so the result looks photographed rather than pasted [4], [21], [22]. Datasets export to YOLO, YOLO-seg [14], and COCO [17] with a re-runnable manifest, and the validation split uses a per-class seed holdout: a source seed reserved for validation never appears in any training image, which closes the train-validation leakage that synthetic data otherwise invites. The whole tool ships as a CLI and a FastAPI web UI; the segmentation tuner, scene preview, and QA montage from that UI appear later in this chapter.

5.3 Sample Screenshots and Output Samples

This section shows the running system and the artefacts it produces. The figures are grouped by surface: the annotated analyze output, the web client, the mobile client, and the MultiSeedGen tool.

The end product of the two-stage pipeline is an annotated image: Figure 5.3 shows the detector's boxes drawn back over a photo, each labelled with the classifier's `good` or `bad` verdict. This is the same overlay the clients render interactively, where a user can filter by quality, raise the confidence threshold, and toggle labels.

[FIGURE PLACEHOLDER]

figures/sample-detection-output.png

Annotated analyze output: detection boxes with per-seed *good*/*bad* quality labels from the classfiier.

Figure :5.3 Annotated analyze output: detection boxes with per-seed `good`/`bad` quality labels from the classfiier.

The web client carries the developer and analyst surface. Figure 5.4 is the Analytics page, which reads windowed trends, confidence distributions, and quality-by-seed-type breakdowns from the ClickHouse warehouse. Figure 5.5 is the Compare view, where two to ten scans are shown side by side with the best column highlighted. Figure 5.6 is the role-gated model registry, where a developer sees each registered artefact, its lifecycle status, and its performance record. Figure 5.7 is the public read-only report produced by the share action, reachable without authentication and localized with its own language and theme toggles.

The mobile client is aimed at farmers in the field. Figure 5.8 shows the scan history on the phone, the same inference graph that the web client reads, rendered for a small screen.

Finally, the MultiSeedGen tool. Figure 5.9 is a generated multi-seed scene with its boxes drawn back from the exported labels, the kind of image the pipeline produces by the thousand. Figure 5.10 is the QA montage the tool writes after a run, a quick visual check that the cut-outs and placements look right before a dataset is used for training. Figure 5.11 is the segmentation

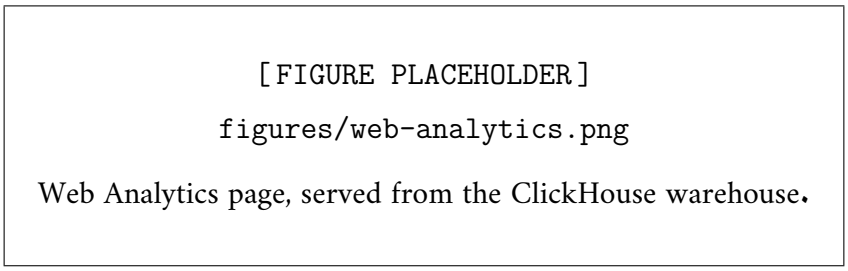

Figure :5.4 Web Analytics page, served from the ClickHouse warehouse.

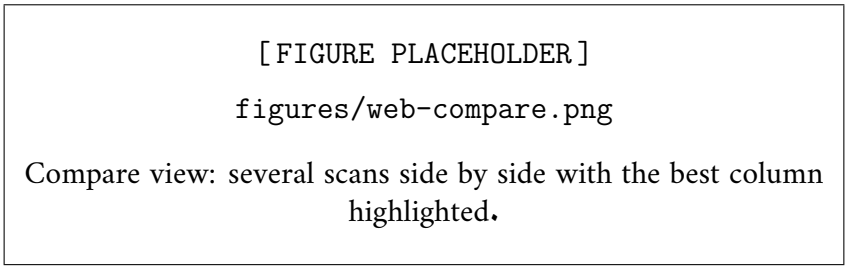

Figure :5.5 Compare view: several scans side by side with the best column highlighted.

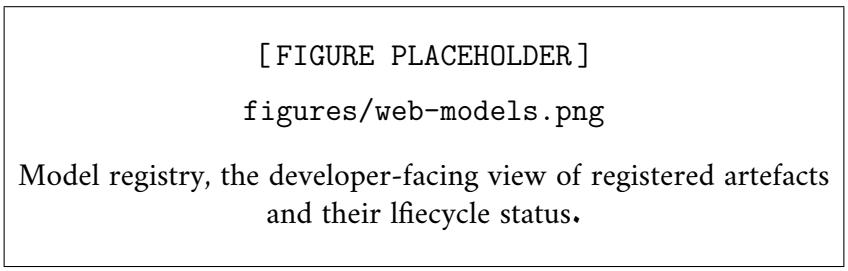

Figure :5.6 Model registry, the developer-facing view of registered artefacts and their lfiecycle status.

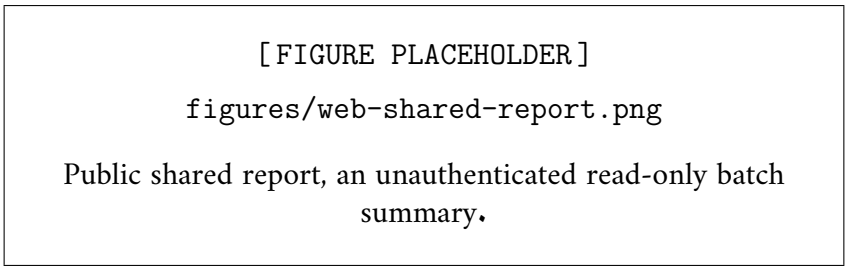

Figure :5.7 Public shared report, an unauthenticated read-only batch summary.

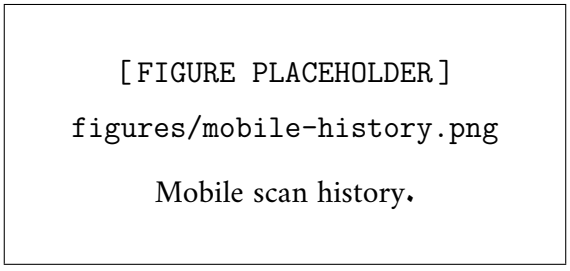

Figure :5.8 Mobile scan history.

tuner from the web UI, which shows kept and skipped cut-outs in a confidence-coded gallery so a user can see exactly which seeds a method dropped and why before committing to a full run.

[FIGURE PLACEHOLDER]

figures/msg-sample-scene.png

A generated multi-seed scene with bounding boxes drawn back from the exported labels.

Figure :5.9 A generated multi-seed scene with bounding boxes drawn back from the exported labels.

[FIGURE PLACEHOLDER]

figures/msg-output-montage.png

MultiSeedGen QA montage, written after a run for a fast visual check of cut-out and placement quality.

Figure :5.10 MultiSeedGen QA montage, written after a run for a fast visual check of cut-out and placement quality.

[FIGURE PLACEHOLDER]

figures/msg-seg-tuner.png

MultiSeedGen segmentation tuner: kept and skipped cut-outs in a confidence-coded gallery.

Figure :5.11 MultiSeedGen segmentation tuner: kept and skipped cut-outs in a confidence-coded gallery.

Chapter 6

Testing and Evaluation

This chapter describes how the system is tested and what the tests tell us about its behaviour. The backend is the largest and most safety-relevant part of the project, so it carries the bulk of the suite, organised as a test pyramid and guarded by automated gates that run on every commit and in continuous integration. The two client applications and the companion data generator, **MultiSeedGen**, each have their own checks tuned to their risks. The chapter closes with what we measured about performance and an honest account of what the suite does not yet cover.

6.1 Testing Strategy

6.1.1 The backend test pyramid

The backend tests are split into tiers by cost and by how much of the real system each tier exercises. The cheap tiers run constantly; the expensive ones run less often but check the parts that unit tests cannot reach.

Unit tests are fast and isolated. They exercise one function or one class at a time with the slow collaborators replaced by fakes: the database session, the object store, and the inference backends are all mocked or stubbed so a unit test never opens a socket. This tier covers the parts of the system that are pure logic and benefit most from quick feedback, including the model resolver's production lookup and global-fallback rules, the batch state-machine transitions, the RFC 9457 error rendering [29], the Pydantic request and response schemas, and the JWT encode and decode paths [28].

Integration tests run against real backing services rather than fakes. The suite uses `testcontainers` to spin up an actual PostgreSQL [31] and ClickHouse [26] for the duration of the run, then tears them down, so repository code is tested against the same database engine it talks to in production rather than against a mock that might disagree with real SQL behaviour. Redis is replaced with `fakeredis` in this tier. Because Celery's `celery_task_always_eager` setting makes tasks execute inline in the same process, the worker pipeline and the OLTP-to-warehouse dual-write can be driven from an integration test without a separate worker

container, which also lets worker-side metrics surface on this path [25].

End-to-end tests drive the full HTTP contract through the ASGI application. They send real requests to the assembled FastAPI app [6] and assert on the response envelopes, status codes, and the RFC 9457 problem bodies, so the routing, middleware, authentication, and serialization are all exercised together exactly as a client would hit them.

On top of the pyramid sits a **smoke test**, `scripts/smoke.sh`, which runs against a live Docker Compose stack [10]. It boots the real services, seeds reference data, registers and promotes a model, then submits an analyze request and polls the batch until it reaches a terminal state. This is the only check that runs the genuine Celery worker with a real PyTorch backend against MinIO, so it is the closest thing in the suite to a production rehearsal.

6.1.2 Pre-commit gates

Before any change reaches a commit, a set of `pre-commit` hooks runs locally. Ruff handles both formatting and linting, so style and a class of likely bugs are caught before review. Mypy runs in strict mode, which means the boundary types between layers are checked statically and a function that returns the wrong shape is rejected at commit time rather than discovered at runtime. Gitleaks scans the staged diff for secrets so that credentials never enter the git history. These same checks form the fast feedback loop that the continuous integration pipeline mirrors.

6.1.3 Continuous integration worfklows

CI is organised as four workflows, each with a clear responsibility, so a red build points at a specific kind of failure.

Table :6.1 Continuous integration worfklows and what each one guards.

Workflow	What it runs and what it protects
`check`	Ruff lint plus mypy strict type-checking plus the unit tier. The fast gate; mirrors the pre-commit hooks so a clean local commit rarely fails here.
`test`	The full pyramid: unit, integration (with `testcontainers` bringing up Postgres and ClickHouse), and end-to-end. This is where database and contract regressions are caught.
`build`	Builds the multi-stage Docker images and enforces the CPU/GPU split. A guard asserts that `torch` is absent from the API image, so the heavy inference dependency can never leak into the lightweight web container.
`smoke`	Brings the Compose stack up, seeds it, and runs `scripts/smoke.sh` end to end against the live services.

The `build` workflow's torch guard deserves a note. The Dockerfile produces separate runtimes so that only the inference worker carries PyTorch and torchvision, while the API and the CPU worker run on a much smaller image. If a careless import ever pulled torch into the API

process, the image would balloon by more than a gigabyte and the architectural boundary would be quietly broken. The CI assertion makes that mistake fail the build rather than ship.

6.1.4 Client and generator testing

The web application is checked with Vitest and Testing Library for unit and component tests, `tsc` for type-checking, and ESLint for linting, with a production `build` as a final gate [8]. The component tests include the localization system, where the test asserts that switching to Arabic flips the document direction to right-to-left, changes the rendered text, and persists the choice. The mobile application, built on Expo and React Native [9], runs `tsc`, `expo-doctor` for project configuration sanity, and a full `expo export` bundle so that a packaging break is caught before a build is cut.

MultiSeedGen adds two kinds of test that matter for a data generator whose whole value is reproducibility. **Golden byte-manifest tests** pin the exact bytes of generated output against committed manifests, so any change that alters a single pixel surfaces as a diff a reviewer must consciously accept rather than a silent regression. **Determinism tests** assert the core contract that a fixed `(config, seed)` pair produces byte-identical output regardless of the parallel backend or the number of worker processes. This is enforced by per-(split, scene-index) random seeding, a content-hashed segmentation cache, and pinning OpenCV to a single thread inside every worker, since OpenCV reductions are not bit-identical across thread counts.

6.1.5 What we report on coverage, and why

The suite is broad. Across the backend there are about **52 test files** and on the order of **293 test cases** spanning the unit, integration, and end-to-end tiers, plus the smoke test and MultiSeedGen's golden and determinism checks. That breadth, together with the CI gates above, is the honest measure of how much of the system is exercised.

We deliberately do not print a single aggregate coverage percentage. The figure recorded in the repository's `coverage.xml` is a non-hermetic measurement artifact: it comes from a fast `check`-style run that does not start the live services the integration and end-to-end tiers depend on, so the router and service code that those tiers do exercise reads as untouched there. Quoting that number would understate the real coverage, sometimes badly. A trustworthy aggregate therefore depends on first making the suite fully hermetic, so that one run can execute every tier and produce one defensible number. That work is tracked as future work and is described in Section 6.4.

6.2 Test Cases and Results

Table 6.2 lists representative cases drawn from across the suite. The *Result* column reports *Pass* where the design and the corresponding tests support the expected behaviour. No numeric accuracy or benchmark figures are claimed here, because the suite asserts behaviour and contracts rather than model accuracy on a held-out set.

Table :6.2 Representative test cases across the system.

ID	Area	Scenario	Expected	Result
T-01	Auth	Login with a correct email and password.	Returns an access token and a refresh token; the login metric records `ok`.	Pass
T-02	Auth	A rotated refresh token is presented a second time (replay).	The reused token is rejected and the token chain is invalidated.	Pass
T-03	Auth	Refresh with a valid current token.	A new access and refresh pair is issued and the old refresh token is retired.	Pass
T-04	Analyze	Multipart upload of several valid images to `POST /analyze`.	A `scan_batch` is created in `pending` and one task per image is dispatched after commit.	Pass
T-05	Analyze	Upload of more than 16 files in one request.	The request is rejected before any image is stored (`analyze_max_files_per_request` is 16).	Pass
T-06	Analyze	Upload of a file whose bytes are not a real image.	Validation fails on the file and no batch is committed.	Pass
T-07	Batch state	Two workers race to start the same pending batch.	A compare-and-set moves `pending` to `running` exactly once; the loser does not corrupt state.	Pass
T-08	Batch state	Detection succeeds but classification raises for an image.	Detection rows are kept and the batch degrades to `partial`, never `failed`.	Pass
T-09	Model resolution	A scan with no seed type is submitted when only a global production model is promoted.	The resolver falls back to the global production model and the batch runs to `succeeded`.	Pass
T-10	Model routing	Analyze runs with no promoted detection model.	The batch ends `failed` with a human-readable message, not a blank error.	Pass

ID	Area	Scenario	Expected	Result
T-11	API keys	A personal API key is created.	The key is shown in plaintext once and stored only as a hash.	Pass
T-12	RBAC	An `end_user` calls an admin-only route.	The request is forbidden by the role gate.	Pass
T-13	DWH	A batch completes with `dwh_enabled` on.	The dual-write task fires and dimension and fact rows reach ClickHouse.	Pass
T-14	MultiSeedGen	Generate twice from one `(config, seed)` with different worker counts.	Both runs produce byte-identical output.	Pass
T-15	MultiSeedGen	Build a train/val split from the seed pool.	The split is leakage-safe; a source seed never lands in both partitions.	Pass
T-16	MultiSeedGen	Export a dataset in YOLO and COCO formats.	The written annotations and `data.yaml` match the pinned export schema.	Pass

A few of these cases are worth expanding because they protect the parts of the system most likely to fail in subtle ways. T-02, the refresh-token replay case, checks that token rotation actually detects reuse rather than merely issuing new tokens; a token presented twice invalidates the whole chain, which is what makes the rotation scheme worth having [28]. T-07 and T-08 protect the batch state machine: because a multi-image batch is processed by several workers at once, the transitions are compare-and-set updates so that exactly one task owns each transition, and a classification crash after a successful detection degrades the batch to `partial` instead of throwing away good detection results. T-10 matters because the mobile point-and-shoot flow sends no seed type and so routes to the global production model; if no model is promoted, the user must see a clear failure rather than an empty screen. T-14 and T-15 are the contracts that make MultiSeedGen useful as a training-data source: reproducible bytes and a split that does not leak the same source seed across partitions.

6.3 Performance Evaluation

The project did not run a formal load-testing campaign; the planned load tests under `tests/load/` were not built, so this section reports the performance behaviour that the design provides and the concrete figures the documentation actually records, rather than synthetic benchmark results.

The dominant cost in the backend is model inference, and the design keeps it off the critical path in two ways. A single image's forward pass runs in a worker thread via `asyncio.to_`

`thread`, so the event loop stays responsive while PyTorch computes. When a batch contains several images, each image is a separate Celery task graded in parallel by the `worker-inference` pool, so a batch is processed concurrently rather than one image at a time, and throughput scales horizontally by adding more inference workers. This is the same path the conveyor-belt scanning use case relies on.

The honest cost is cold-start CPU inference, which is slow: the first inference on a cold CPU process pays the model load and the first forward pass with no warm caches. The smoke test allows roughly 75 seconds for a batch to complete on a cold CPU, which sets the expectation for an un-warmed deployment. After the first run, the process-wide LRU cache in the model manager keeps loaded weights resident, so subsequent inferences skip the load cost and run far faster. On the observability side, Prometheus histograms expose both HTTP request latency and per-image inference latency, so latency can be tracked per route and per model backend once a deployment is running [30].

For MultiSeedGen, the documentation records two concrete effects. Running the rembg background-removal model on a GPU is roughly an order of magnitude faster than on CPU, measured at about 11x on an RTX 3060 [19]. Separately, the content-hash segmentation cache makes re-runs near-instant: once a source seed has been segmented, a later run with the same inputs reuses the cached result instead of recomputing it. These figures are collected in Table 6.3.

Table :6.3 Observed and expected performance figures recorded in the project documentation.

Aspect	Figure	Source
Cold-start CPU batch completion	$\approx$ 75 s allowed	Smoke-test budget
Warm inference	Much faster after first load	Model-manager LRU cache
MultiSeedGen rembg, GPU vs CPU	$\approx$ 11x faster on an RTX 3060	MultiSeedGen docs
MultiSeedGen cached re-run	Near-instant	Content-hash segmentation cache

6.4 Limitations and Known Issues

The system works end to end, but the testing and evaluation tail of the project is unfinished, and several known gaps are worth stating plainly.

The most consequential testing limitation is that the suite is **not fully hermetic**. Some integration tests boot the assembled application, and the slowapi rate limiter inside that app reaches for a live Redis on the Compose hostname, which `fakeredis` does not intercept; outside Docker those tests fail for an environmental reason rather than a product bug [27]. The direct conse-

quence is the one discussed in the testing strategy: until the rate limiter runs on fakeredis end to end, or CI provides a real Redis service for the test job, there is no single coverage number that can be trusted. Making the suite hermetic and then establishing a defensible aggregate coverage figure is the first item of remaining work.

On the product side, image upload is **multipart-only**. The presigned-upload endpoint and batch cancellation were designed but never built, so a client cannot upload directly to object storage or abort a running batch. The analytics warehouse is populated by an **application-level dual-write** rather than true change-data-capture: when a batch completes, the application dispatches a Celery task that writes to ClickHouse, instead of streaming changes off the Postgres write-ahead log. The database is configured with logical replication enabled so that real CDC can be added later, but that path is not wired today.

Observability is **opt-in**. OpenTelemetry tracing and Sentry error reporting are complete no-ops unless their endpoints are configured, which keeps the development stack quiet but means neither is exercised by default. A handful of endpoints have a service and a schema behind them but no route yet, including password change and experiment-results retrieval, so the underlying capability exists but is not reachable over HTTP.

Finally, a limitation specific to MultiSeedGen's purpose: there is a **synthetic-to-real domain gap**. Compositing seeds onto backgrounds with matched lighting and camera degradation produces realistic-looking images, but a detector trained only on synthetic data can still behave differently on real field photographs than its synthetic validation metrics suggest [4]. The generator narrows the gap with domain-matched degradation, but closing it for a given crop still requires validating the trained detector against genuine images before trusting it in the field [1].

Chapter 7

Conclusion and Future Work

This chapter closes the report. It first restates what was actually built and maps it back to the objectives set out in Chapter 1, then records the lessons that came out of building it, and finally lays out the work that would take the system from a working platform to something a seed cooperative could run in production for a season.

7.1 Summary of Achievements

The project set out to build a seed-quality intelligence platform: a user photographs a batch of seeds, and the system detects each individual seed and classifies its quality, then reports aggregate metrics such as seed count, good-rate, and a per-crop breakdown. That platform exists and runs. What follows is an account of the pieces that were delivered, organised around the objectives from Chapter 1.

7.1.1 An end-to-end, traceable inference platform

The core of the product is a two-stage vision pipeline. A detection model scans the whole image and returns a bounding box, a confidence, and a crop class for every seed it finds; then each detected box is cut from the source image and fed to a binary classifier that returns `good` or `bad` with a confidence [1]. The detector is a torchvision Faster R-CNN [11] with a ResNet-50 backbone and a Feature Pyramid Network [12], [13], carrying a three-class head over `background`, `coffee`, and `maize`. The classifiers are ResNet-18 backbones with a Convolutional Block Attention Module inserted after the final residual stage [18], one classifier trained per crop type. Splitting the problem this way means a detector and a classifier can be versioned and swapped independently, because detection and classification are recorded as separate `inferences` rows over the same image.

Traceability was a hard requirement rather than a nice-to-have, and it is enforced at the schema level. Every `seed_detection` row carries a foreign key to the `inference` that produced it, and every `inference` carries a foreign key to the exact `model_artifact` that ran.

Any single seed label in the system can be traced back to the model version and weights that generated it. This chain is what makes the rest of the ML platform trustworthy: an offline evaluation score means nothing if you cannot say which weights produced which label.

The request path that ties these together is the unified `POST /analyze` endpoint. An upload lands in MinIO and Postgres, a Celery task picks it up [25], the `ModelResolver` selects a model for the segment, the detector runs, each crop is classified, and the results are persisted with the traceability chain intact. Batch state moves through a state machine (`pending` → `running` → `succeeded/partial/failed`), and clients poll `GET /batches/{id}` for progress. The failure handling is deliberate: if classification crashes after detection has already persisted, the batch degrades to `partial` rather than discarding good detection results, and if no model is promoted the user gets a readable message instead of a blank error.

7.1.2 A model registry with production-model resolution and offline evaluation

Weights are uploaded to MinIO and recorded in the `model_artifacts` table, each row carrying its kind, backend, builder key, a per-model config blob, and a lifecycle status that moves `registered` → `staging` → `production` → `archived`. Only staging and production models can serve traffic. The `ModelResolver` decides which model runs for a given segment: it resolves the production model promoted for that crop type, falling back to a global production model when no per-type model exists, so a single promoted model can serve every scan. That fallback is what lets the mobile point-and-shoot flow, which sends no crop type, still resolve to a model.

Around the registry sits the experiments subsystem. A labelled dataset stored in MinIO and Postgres is the ground truth; an experiment runs a chosen model over that dataset offline, computes classification metrics including the confusion matrix, persists its metrics to PostgreSQL and writes a Markdown report to object storage. The model manager keeps a per-process LRU cache of loaded modules, serialises concurrent loads of the same model through a lock so two workers never duplicate a GPU load, and hot-reloads a model when its artifact timestamp advances. A developer can register new weights, evaluate them offline, and promote them to production so they serve live scans without a restart.

7.1.3 Three clients, bilingual and right-to-left

The backend serves one API to three clients. The first client is the HTTP API itself, a versioned, layered FastAPI service [6], [32] with JWT and OAuth2 authentication [24], [28], role-based access control, per-route rate limiting, and RFC 9457 problem-details errors [29]. The second is a React web application [8] covering the farmer journey of sign in, photograph, report, and history, plus the developer-facing model and analytics pages. The third is an Expo / React Native

mobile app [9] with a realtime camera capture flow. Both user-facing clients ship in Arabic and English with full right-to-left layout, which matters for the intended Egyptian farming audience and is not something that can be retrofitted cheaply.

7.1.4 Observability and operations

The platform is instrumented rather than opaque. It exposes Prometheus metrics, a Grafana dashboard set, OpenTelemetry tracing [30], Sentry error reporting, structured logging, and `/healthz` and `/readyz` probes. Configuration comes from one central `Settings` object sourced from environment variables or file-based secrets, never read piecemeal from the process environment. The whole stack ships as a multi-stage Docker build [10] with a development compose file and a production overlay. ClickHouse [26] backs the analytics pages through a star schema so heavy analytical queries stay off the OLTP Postgres [31].

7.1.5 MultiSeedGen: closing the labelling gap

The companion tool, MultiSeedGen, addresses a problem that blocks the detector more than any architectural choice does. Training an object detector needs images of *many* seeds with a bounding box drawn around each one, and labelling those boxes by hand is slow and error-prone. What is comparatively easy to obtain is photographs of single seeds. MultiSeedGen turns the cheap input into the expensive output: it segments individual seeds out of source photos using a cascade of classical masks and learned matting via rembg and an optional Segment Anything backend [19], [20], then composites many of those cutouts onto realistic backgrounds and exports a detection dataset in YOLO, YOLO-seg, and COCO formats with the bounding boxes generated for free as a by-product of placement [3], [15], [17].

Two design decisions make the output usable for training rather than a toy. First, the compositor applies domain-matched lighting and camera degradation, a deliberate move toward closing the gap between synthetic and real images through domain randomisation and augmentation [4], [21], [22]. Second, output is byte-reproducible for a fixed configuration and seed, so a dataset is defined entirely by its recipe; an experiment can be rerun exactly, which is the same reproducibility discipline the inference platform enforces through its traceability chain. The tool ships with a CLI and a FastAPI web UI over the same pipeline, and writes provenance and QA reports alongside every run.

Taken together, the two systems form a loop on paper: MultiSeedGen produces the labelled detection data the platform's detector is trained on, and the platform produces the predictions that, in the future, can tell MultiSeedGen which kinds of scenes it is generating too few of. Section 7.3 returns to closing that loop in practice.

7.2 Lessons Learned

The technical lessons that came out of this project were mostly about discipline paying off slowly, and shortcuts charging interest later.

The strict layered architecture earned its keep. Holding the line on `routers` → `services` → `repositories` → ORM [5], with Pydantic schemas at every boundary and the ORM never leaking past the service layer, sounded like ceremony at the start. Its value showed up when the ML platform grew. Because every backend satisfies the same duck-typed protocol and returns framework-free DTOs, adding the YOLO and Roboflow backends alongside the default local-torch one was a matter of writing a class and registering it in a factory [33], with no churn in the services that call them. The same payoff showed up in model selection: collapsing the routing layer down to a deterministic, stateless `ModelResolver` that resolves the promoted production model for a segment, with a global fallback, left the services that call it untouched and removed an entire stateful subsystem from the request path.

The single `Settings` object was a similar bet that paid off. Routing all configuration through one validated object, sourced from environment variables or file secrets, meant the production compose overlay could swap in file-based secrets without touching application code, and it removed the class of bug where two modules read the same setting differently. It is a small rule, applied everywhere, and the payoff is that there is exactly one place to look.

Keeping torch out of the API image turned out to matter more than expected. The heavy imports for torch and ultralytics live inside method bodies in the inference backends, so the API process never imports torch; only the inference worker does. The benefit is a small API container that starts fast and a clean separation between the web tier and the GPU tier. The cost is that this invariant is easy to break by accident with a careless top-level import, which is why one of the planned CI checks is an explicit assertion that the API image contains no torch [7].

The most expensive lesson was about tests. The suite is not hermetic: the app-booting integration tests fail outside Docker because the rate limiter dials a real Redis at the compose hostname, and the fakeredis fixture does not intercept that client [27]. Without continuous integration to catch it, the suite rotted quietly. A fresh run surfaced two stale assertions that referenced renamed internals, and 13 of the integration tests failed for this Redis dependency rather than any product defect. The honest position is that the committed coverage number is a partial artifact and cannot be trusted until the suite runs the same way everywhere. Non-hermetic tests do not announce themselves; they pass on the author's machine and quietly stop meaning anything.

The synthetic-to-real domain gap is real and was treated as such. Generating plausible-looking multi-seed images is the easy part. Generating images close enough to real field photographs that a detector trained on them transfers cleanly is the hard part, which is exactly why MultiSeedGen invests in camera-matched degradation and domain randomisation rather than clean composites. Synthetic data narrows the labelling bottleneck; it does not erase the distribu-

tion shift, and the right way to measure whether it has helped is to evaluate the detector against real labelled scans, not against more synthetic data.

Finally, coordinating a backend, two frontends, and a separate data tool was a project-management lesson as much as a technical one. Two of the three planned clients, two crop classifiers, a detector, and a generator each move at their own pace, and the only thing that kept them coherent was treating the API contract and the data formats as the fixed interfaces and letting everything else vary behind them. The places where that discipline slipped, such as documentation pointing at files that had moved and a script referenced in the README that was never written, are precisely the places where the contract was implicit rather than enforced.

7.3 Future Enhancements

The work below is ordered roughly by how much it unblocks. The first item is a precondition for trusting almost everything else.

Make the test suite hermetic and publish a real coverage number. The integration tests must run identically on a developer laptop and in CI. That means making the application client use fakeredis end to end, or adding a Redis service to the test compose file, so the rate limiter has something to dial without a live cluster. A dependency lockfile should be added in the same pass so installs are reproducible and library drift stops silently breaking the build. Only after that is continuous integration worth standing up, and only then is a coverage number meaningful enough to put in a report. The worst-covered areas are already known: the worker tasks and the ML backends are barely exercised, and MinIO storage and the OAuth callback are mocked everywhere.

Presigned and resumable uploads, and batch cancellation. Inference uploads are multipart-only today. Large field photographs and slow rural connections argue for presigned uploads that go straight to MinIO, with resumable transfers so a dropped connection does not restart a multi-megabyte upload from zero. The service and schema for presigned upload already exist; what is missing is the route. Batch cancellation belongs in the same effort: a user who realises they uploaded the wrong batch should be able to stop a running scan rather than wait for it to finish.

True change-data-capture to the warehouse. The ClickHouse warehouse is populated by an application-level dual-write: when a fact is written to Postgres, a Celery task also writes it to the star schema. This works but couples analytics correctness to application code and can drift if a write path is added without a matching dual-write. Postgres is already configured for logical replication; the enhancement is to consume that replication stream and let change-data-capture populate the warehouse, so analytics derive from the source of truth instead of a parallel write that has to be kept honest by hand.

More crop types. The platform supports coffee and maize today, and the architecture was built to extend. A new crop is a new detector head class, or a retrained detector, plus a new per-crop classifier registered through a builder and the model registry. The `ModelResolver` already resolves models per crop type, so a new classifier can be promoted and swapped in isolation without disturbing the existing crops. On the data side, MultiSeedGen extends the same way: a new crop is a new set of source seed photos and a config recipe, with no pipeline changes.

On-device inference for the mobile app. The mobile app currently captures images and sends them to the backend for inference. For field use with poor connectivity, running a lightweight detector and classifier on the device would let a farmer get a result with no round trip. This is a natural fit for a quantised YOLO detector [14], [15] exported to a mobile runtime, with the server remaining the authority for history and the traceability record once connectivity returns.

An active-learning loop. The most valuable enhancement closes the loop between the two systems. Real scans that the platform classifies with low confidence, or that a user corrects, are exactly the cases the current models handle worst, and they describe the scenes the synthetic generator is underproducing. The loop is: collect those hard real scans through the existing traceability chain, feed their characteristics back into MultiSeedGen to regenerate training data weighted toward the failure modes, retrain the detector and the affected classifier on the augmented set, evaluate the candidate offline against a held-out set of real labelled scans, and promote the improved model to production through the registry's existing staging-to-production lifecycle once it clears that bar. Each turn of the loop targets generation at the platform's measured weaknesses instead of producing more of what it already handles well. The registry, the offline-evaluation runner, the `ModelResolver`, and MultiSeedGen's reproducible recipes are all already in place; what remains is the orchestration that connects them.

References

[1] A. Kamilaris and F. X. Prenafeta-Boldú, ``Deep learning in agriculture: A survey,'' *Computers and Electronics in Agriculture*, vol. ,147 pp. –70,90 .2018

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, ``Using deep learning for image-based plant disease detection,'' *Frontiers in Plant Science*, vol. ,7 p. ,1419 .2016

[3] D. Dwibedi, I. Misra, and M. Hebert, ``Cut, paste and learn: Surprisingly easy synthesis for instance detection,'' in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, ,2017 pp. –1301.1310

[4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, ``Domain randomization for transferring deep neural networks from simulation to the real world,'' in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, ,2017 pp. –23.30

[5] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, .2017

[6] S. Ramírez, ``Fastapi framework, high performance, easy to learn, fast to code, ready for production,'' .2018 [Online]. Available: https://github.com/tiangolo/fastapi.

[7] M. Bayer, *SQLAlchemy: Object Relational Tutorial*. SQLAlchemy Org, .2023

[8] M. Go, *React 18 Design Patterns and Best Practices*. Packt Publishing, .2022

[9] E. Team, *Expo SDK :52 Cross-Platform Native Mobile Development*. O'Reilly Media, .2024

[10] D. Merkel, ``Docker: Lightweight linux containers for consistent development and deployment,'' *Linux Journal*, vol. ,2014 no. ,239 .2014

[11] S. Ren, K. He, R. B. Girshick, and J. Sun, ``Faster R-CNN: towards real-time object detection with region proposal networks,'' *CoRR*, vol. abs/1506.01497, .2015 [Online]. Available: http://arxiv.org/abs/1506.01497.

[12] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, ``Feature pyramid networks for object detection,'' in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, ,2017 pp. –2117.2125

[13] K. He, X. Zhang, S. Ren, and J. Sun, ``Deep residual learning for image recognition,'' *CoRR*, vol. abs/1512.03385, .2015 [Online]. Available: http://arxiv.org/abs/1512.03385.

[14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, ``You only look once: Unfiied, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, ,2016 pp. –779.788

[15] G. Jocher, A. Chaurasia, and J. Qiu, ``Ultralytics yolov8," .2023 [Online]. Available: https://github.com/ultralytics/ultralytics.

[16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, ``ImageNet: a large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, ,2009 pp. –248.255

[17] T.-Y. Lin, M. Maire, S. Belongie, *et al.*, ``Microsoft COCO: common objects in context," in *Proceedings of the European Conference on Computer Vision (ECCV)*, ,2014 pp. –740.755

[18] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, ``CBAM: convolutional block attention module," in *Proceedings of the European Conference on Computer Vision (ECCV)*, ,2018 pp. –3 .19

[19] X. Qin, Z. Zhang, C. Huang, M. Dehghan, O. R. Zaiane, and M. Jagersand, ``U2-Net: going deeper with nested u-structure for salient object detection," *Pattern Recognition*, vol. ,106 p. 107 ,404 .2020

[20] A. Kirillov, E. Mintun, N. Ravi, *et al.*, ``Segment anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, ,2023 pp. –4015.4026

[21] C. Shorten and T. M. Khoshgoftaar, ``A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. ,6 no. ,1 pp. –1,48 .2019

[22] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, ``Albumentations: Fast and flexible image augmentations," *Information*, vol. ,11 no. ,2 p. ,125 .2020

[23] M. Zaharia, A. Chen, A. Davidson, *et al.*, ``Accelerating the machine learning lfiecycle with mfllow," in *IEEE International Conference on Big Data*, .2018

[24] D. Hardt, *The OAuth* 2.0 *authorization framework*, RFC ,6749 Internet Engineering Task Force, .2012 [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749.

[25] D. R. Hall, *Distributed Task Queuing with Celery*. Packt Publishing, .2016

[26] Y. Bezrukov, ``Clickhouse: An open-source column-oriented database management system for online analytical processing (olap)," *Journal of Database Management*, .2020

[27] J. L. Carlson, *Redis in Action*. Manning Publications, .2013

[28] M. B. Jones, J. Bradley, and N. Sakimura, *JSON web token (JWT)*, RFC ,7519 Internet Engineering Task Force, .2015 [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519.

[29] M. Nottingham, E. Wilde, and S. Dalal, *Problem details for HTTP APIs*, RFC ,9457 Internet Engineering Task Force, .2023 [Online]. Available: https://www.rfc-editor.org/rfc/rfc9457.

[30] T. Morgan, ``Distributed tracing and observability with opentelemetry," *IEEE Software*, .2022

[31] D. Fontaine, *The Art of PostgreSQL*. PostgresPress, .2020

[32] R. T. Fielding, ``Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of Calfiornia, Irvine, .2000

[33] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns*: *Elements of Reusable Object-Oriented Software*. Addison-Wesley, .1994

Appendix A

API Endpoint Reference

The Seed Bank backend exposes a single versioned REST surface mounted under `/api/v1` [32]. Routers are split by resource, and the interactive OpenAPI documentation is served at `/api/v1/docs` with the raw schema at `/api/v1/openapi.json`; the web client's typed request layer is generated from that schema, so the table below and the running code cannot drift apart. Every successful single response is wrapped in a `{ "data": ... }` envelope, list responses add a `meta` block with pagination, and every error is an RFC 9457 problem-details document carrying the request's `request_id` [29].

Table A.1 lists the representative endpoints grouped by router. Paths are shown relative to the `/api/v1` prefix. Authentication uses a short-lived JWT access token [28], except for the public `shared` route and the OAuth handshake [24].

Table A.1: Representative REST endpoints by router (paths relative to `/api/v1`.(

Method	Path	Purpose
auth		
POST	`/auth/register`	Create a local account; sends cation email.
POST	`/auth/verify-email`	Confirm an account from an token.
POST	`/auth/login`	Exchange email and passwor cess + refresh tokens.
POST	`/auth/refresh`	Rotate the refresh token and new access token.
POST	`/auth/logout`	Revoke the current refre chain.
GET	`/auth/oauth/{provider}/login`	Start a Google or GitHub OA
GET	`/auth/oauth/{provider}/callback`	Complete the OAuth redirect a tokens.
POST	`/auth/bootstrap-admin`	One-shot first-admin guarded by a shared token.
users		

continued on

Table A.1 continued from previous page

Method	Path	Purpose
GET	`/users/me`	Return the authenticated user's
GET	`/users`	Admin: list users with pagina
PATCH	`/users/{user_id}/role`	Admin: change a user's role.
api_keys		
POST	`/api-keys`	Create a personal key (retu plaintext once).
GET	`/api-keys`	List the caller's keys (metadat
DELETE	`/api-keys/{key_id}`	Revoke a key.
models		
POST	`/models`	Register a model artifact weights/metadata.
GET	`/models`	List registered models.
GET	`/models/{model_id}`	Fetch metadata for a specific r
PATCH	`/models/{model_id}`	Advance lifecycle status ing/production/archived).
GET	`/models/{model_id}/performance`	Per-model offline metrics an House analytics.
analyze		
POST	`/analyze`	Multipart image upload that inference batch.
batches		
GET	`/batches`	List the caller's batches.
GET	`/batches/{batch_id}`	Poll one batch for status and r
GET	`/batches/{batch_id}/image-urls`	Fetch short-lived presigned U batch's images.
POST	`/batches/compare`	Compare aggregate stats side for 2–10 batches.
DELETE	`/batches/{batch_id}`	Soft-delete one batch.
POST	`/batches/delete`	Soft-delete up to 200 owned ba bulk.
GET	`/batches/{batch_id}/export.csv`	Export all detections in the CSV.
GET	`/batches/{batch_id}/export.json`	Export all detections in the JSON.
GET	`/batches/{batch_id}/images/{image_id}/annotated.png`	Get original image with d boxes burned in.
POST	`/batches/{batch_id}/share`	Create or rotate public read-on link.
DELETE	`/batches/{batch_id}/share`	Revoke the share link.
analytics		
GET	`/analytics`	Aggregated metrics over a ti dow (ClickHouse).

continued on n

Table A.1 continued from previous page

Method	Path	Purpose
shared		
GET	`/shared/{token}`	Public, unauthenticated r batch report.
catalog		
GET	`/seed-types`	Reference list of seed types.
GET	`/suppliers`	Reference list of visible suppl
POST	`/suppliers`	Create a global (admin) or priv plier.
PATCH	`/suppliers/{supplier_id}`	Update a supplier's metadata status.
DELETE	`/suppliers/{supplier_id}`	Soft-delete a supplier.
datasets		
POST	`/datasets`	Create a labelled dataset fo evaluation.
GET	`/datasets`	List datasets.
GET	`/datasets/{dataset_id}`	Fetch metadata for a specific
POST	`/datasets/{dataset_id}/items`	Add ground-truth items to a d
GET	`/datasets/{dataset_id}/items`	List items in a dataset with pag
experiments		
POST	`/experiments`	Create and run an offline-ev experiment.
GET	`/experiments`	List experiments.
GET	`/experiments/{experiment_id}`	Fetch experiment status and su

Appendix B

Requirements Traceability

The consolidated functional and non-functional requirements are defined once, in Chapter 3. The use-case diagram appears as Figure 3.1, and the functional (FR) and non-functional (NFR) requirement identifiers are listed in the FR and NFR tables of that chapter. To avoid two copies that can disagree, they are not reprinted here.

For grading and review, the mapping from requirement to implementation is direct. Each FR resolves to one router in Appendix A: the analyze and batch requirements map to the `analyze` and `batches` routers, the model-management requirements map to the `models` router, and the identity requirements map to `auth`, `users`, and `api_keys`. The data-traceability requirement maps to the four-table inference chain (`scan_batches` $\rightarrow$ `scan_images` $\rightarrow$ `inferences` $\rightarrow$ `seed_detections`) described in Chapter 4. The non-functional requirements map to mechanisms documented elsewhere in the report: concurrency safety to the compare-and-set batch state machine (Appendix D), observability to the Prometheus, OpenTelemetry, and Sentry signals, and reproducibility on the synthetic-data side to the seeded MultiSeedGen pipeline (Appendix C).

Appendix C

MultiSeedGen Configuration Reference

MultiSeedGen is the companion tool that turns single-seed source photos into multi-seed detection datasets by cut-and-paste compositing [3]. It is driven the same way from a CLI flag, a YAML/JSON config file, or the web form, because all three feed one validated `Config`; CLI flags override file values, and every run writes a `run_config.yaml` that reproduces the dataset when fed back in. The recommended workflow is to verify segmentation first, generate, then validate the exported labels.

Table C.1 lists the knobs that matter most in practice, with their defaults. Quality of the cut-outs and the domain match of the backgrounds dominate dataset quality far more than image count, so the segmentation and background flags are the ones worth tuning first.

Table C.1: Most useful MultiSeedGen configuration flags and their effect.

Flag	Default	Effect
`--segment`	`auto`	Cut-out method: `auto` (classical cascade with a learned fallback), `rembg` (U2-Net matting [19]), `grabcut`, `otsu`, `u2net`.
`--seg-conf-thresh`	`0.55` / `0`	Skip-gate for low-confidence cut-outs. Active in `auto`; opt-in (defaults to `0`, off) for explicit methods.
`--rembg-provider`	`auto`	Run the learned matting on GPU when present (about $11\times$ faster), CPU otherwise.
`--min-seeds`	n/a	Lower bound on seeds composited per scene.
`--max-seeds`	n/a	Upper bound on seeds per scene (must be $\geq$ `min-seeds`).
`--scale-min`	n/a	Smallest per-seed scale as a fraction of the canvas.
`--scale-max`	n/a	Largest per-seed scale (controls apparent seed size).

continued on next page

Table C.1 continued from previous page

Flag	Default	Effect
`--bg-from-sources`	on (rec.)	Composite onto the real tray inpainted from sources; closes the domain gap [4].
`--neg-frac`	`0.10`	Fraction of background-only scenes to suppress false positives.
`--shadow-dir-mode`	`scene`	One light direction per scene for soft, grounded shadows; `--no-shadow` disables.
`--shadow-opacity`	`0.25`	Strength of the drop shadow under each seed.
`--edge-feather`	`0.5`	Anti-aliases the paste edge only; never moves labels.
`--degrade`	on (mild)	Adds sensor noise, JPEG artefacts, and slight defocus so composites read as photographed [21].
`--noise-sigma`	`3`	Gaussian sensor-noise sigma for the degrade pass.
`--jpeg-quality-min`	`80`	Lower bound of randomized JPEG re-encode quality.
`--jpeg-quality-max`	`95`	Upper bound of randomized JPEG re-encode quality.
`--blur-max-sigma`	`0.5`	Maximum defocus blur applied during degrade.
`--perspective`	`0`	Per-seed perspective warp before placement; `0.05`–`0.1` adds viewpoint variety.
`--class-mode`	`single`	`subfolders` maps each top-level source folder to a class; `single` merges all to one.
`--fmt`	n/a	Export format: `yolo`, `coco`, `yolo-seg` (polygons), or `both` [15], [17].
`--masks`	off	Also write per-instance masks alongside boxes.
`--num-images`	n/a	Number of synthetic scenes to generate (`0` = verify only).
`--img-size`	n/a	Output canvas size in pixels.
`--val-split`	n/a	Fraction of generated scenes held out as the validation split.
`--workers`	0 (auto)	Parallel scene generation; output is byte-identical regardless of worker count.
`--parallel-backend`	`auto`	`loky` processes when assets fit in RAM, else falls back to `threading` to avoid OOM.

continued on next page

Table C.1 continued from previous page

Flag	Default	Effect
`--seed`	7 (rec.)	Random seed; a fixed seed gives byte-identical output.
`--config`	n/a	Load a YAML/JSON baseline; CLI flags still override it.
`--cache-dir`	`<out>/.seg_cache`	Content-hashed segmentation cache shared across runs.
`--qa-report`	on (rec.)	Emit QA charts and a visual preview to eyeball before training.

A caveat carried into this report: the bundled `seeds/Test` set is single-seed, so it validates the classifier and the photometric domain but not multi-seed detection. Synthetic data is for training; a small real, hand-labelled multi-seed set is kept as the evaluation target.

Appendix D

Source Code Excerpt

The batch state machine is the part of the orchestration that has to stay correct under concurrency: a multi-image batch fans out into one Celery task per image, and several workers can run at once. Each batch transition (`pending` → `running` and `running` → terminal) is therefore a compare-and-set update, not a read-then-write. The transition only commits if the row is still in the expected state, so exactly one task wins each transition and the others see zero rows affected and step aside. The excerpt below is a sketch of that update; the real repository method also stamps `started_at` and emits the dual-write to ClickHouse.

```
async def try_transition(session, batch_id, expected, new_status):
    # Compare-and-set: only the first task to arrive flips the state.
    stmt = (
        update(ScanBatch)
        .where(ScanBatch.id == batch_id)
        .where(ScanBatch.status == expected)   # the "compare"
        .values(status=new_status)             # the "set"
        .returning(ScanBatch.id)
    )
    result = await session.execute(stmt)
    won = result.scalar_one_or_none() is not None
    await session.commit()
    return won   # True  -> this task owns the transition
                 # False -> another worker already moved the batch
```

The first task to flip `pending` → `running` owns the start of the batch; the task that records the last image's detection inference flips `running` to one of `succeeded`, `partial`, or `failed`. Because the `WHERE` clause carries the expected status, the database serializes the contest and no batch can be advanced twice.